
NiCE Search Grid Administration Guide Version 20.1.2.1

Administrator's Introduction to NiCE Search Grid™

Search

Type	Description
Conventional Search	Conventional search is Search Grid's standard phonetic search technique. Compared to large scale search, conventional search is slower, but offers higher recall. Also, indexing for conventional search requires less CPU time and storage space.
Large Scale Search (LSS)	LSS is the fastest search technique that Search Grid supports. LSS is orders of magnitude faster than conventional search, but its indexes require more CPU time to create and more space to store. Also, LSS provides less recall than conventional search; that is, LSS is less likely to find phrases that are spoken less distinctly.

Phonetic Indexes and Models

Type	Description
Pat	Also called phonetic file or phonetic access track file, pats are indexes that support phonetic search. Creating a pat is the first step in phonetic indexing. Both conventional search and LSS require a pat to be available.
Phonetic Digest	A kind of phonetic index used to support LSS. Search Grid creates two varieties of phonetic digest: monogests and polygests.

Type	Description
Monogest	A kind of phonetic digest related to a single piece of media. Monogests are created from pats and aggregated into polygests. Search Grid automatically creates monogests from pats and aggregates them into polygests. Search Grid users do not directly interact with monogests (as they do with pats), but administrators may need to consider monogest storage during configuration.
Polygest	A kind of phonetic digest related to multiple pieces of media. Search Grid automatically creates, manages, and deletes polygests to support LSS. Search Grid users do not directly interact with polygests (as they do with pats), but administrators may need to consider polygest storage and creation criteria during configuration.
Language Pack	A language pack comprises a set of phonetic model files that allow Search Grid to perform phonetic analysis. Language packs contain information about a spoken language and the media used to record the language. Language packs are distributed separately from Search Grid, and may be installed via Management Console or programmatically by a client application.
Phrase Pack	A phrase pack contains a set of model files that support Search Grid's phrase recognition feature. Like language packs, phrase packs are distributed separately from Search Grid and must be installed via Management Console or programmatically by a client application; however, a major difference from language packs is that a phrase pack is associated with a specific tenant rather than being globally accessible resources. This is because phrase packs are commonly trained for a specific user or customer. Another difference from language packs is size. A phrase pack consumes about 2 GiB on each Data Node, compared to a few hundred megabytes for language packs.

Type	Description
Search Model	An optional phonetic model file that increases the quality of search results. Some language packs include a search model. For other language packs, the search model is installed separately.
Phonetic Digest Model	An optional phonetic model file that enables the creation of phonetic digests (i.e., monogests and polygests). Some language packs include a phonetic digest model. For other language packs, the phonetic digest model is installed separately.
Language Identification Model	A language identification model enables Search Grid to determine the language used in recorded audio. Search Grid includes two language identification models for demonstration purposes. More accurate models can be created with NiCE's AudioFinder application, or through an engagement with NiCE Professional Services.
Classifier Model	A classifier model allows Search Grid to identify different types of audio features in recorded media. Recognized features include speech, gender, and DTMF tones.

Nodes

Search Grid is organized into fundamental deployment units called grid nodes or simply nodes. There are several types of nodes (described below), each type performing a different set of system functions. A typical system may include multiple servers, each server hosting one or more nodes.

Type	Description
Control Node	Control Nodes store global configuration data and coordinate distributed activities. A Search Grid deployment must have either one or three Control Nodes. Using three Control Nodes enables redundant, self-repairing data storage and high availability. A set of three Control Nodes is often called an <i>ensemble</i> . The

Type

Description

minimum number of Control Nodes required, or the *quorum*, is a simple majority of those deployed. So if three Control Nodes are deployed, at least two must be running to function. (If just one Control Node is deployed, the quorum is one.)

NOTE: High availability of Control Nodes is separate from high availability of Data Nodes for media declarations. (See the [Large Distributed Deployment](#) section.)

Gateway Node

Gateway Nodes host the grid's primary web servers. All client HTTP interactions start with a Gateway Node, although the Gateway may redirect the client to other nodes. Gateway Nodes store no persistent data. Each Search Grid deployment must contain one, two, or three Gateway Nodes. Deploying more than one Gateway Service facilitates failover and load balancing between them. At least one Gateway Service must be running for Search Grid to function.

Data Node

Data Nodes store, organize, and search phonetic indexes. Data Nodes also perform CPU-intensive operations like phonetic index creation, classification, and language identification. A Search Grid deployment must have at least one Data Node, but using multiple Data Node servers can increase search performance. When using multiple Data Nodes, each node stores information about a subset of the media in the system. Defining the number and identity of the Data Nodes is an important part of Search Grid sizing and configuration. Data Nodes can be added or removed after the initial deployment by following the procedures described in the Data Redistribution section. When there are multiple Data Nodes, Search Grid can be optionally configured to enable high availability for media declarations.

NOTE: High availability of Data Nodes for media

Type	Description
	declarations is separate from high availability of Control Nodes. (See the Large Distributed Deployment section.)
Compute Node	Compute Nodes, like Data Nodes, perform CPU-intensive operations like phonetic index creation, classification, language identification, and certain kinds of search. Unlike Data Nodes, however, Compute Nodes do not store data locally. Compute Nodes can be added or removed after the initial deployment with no reconfiguration and no system downtime. This facilitates dynamically scaling out a Search Grid installation to add or remove CPU resources. A Compute Node, however, should never be installed on the same server as another node type.

Services

Each node comprises one or more services visible in the Services control panel. Control, Gateway, and Compute Nodes have one service each (the Control, Gateway, and Compute Services, respectively) that perform all the functions for that node. Data Nodes comprise the following two services:

Type	Description
Base Service	The Base Service organizes phonetic index files for the node and enables fast retrieval at search time. The Base Service is also the target for client HTTP requests redirected from Gateway to the Data Node. Other services in the Data Node rely on the Base Service for communication and coordination; they cannot function unless the Base Service is running.
Agent Service	The Agent Service performs phonetic searches for both conventional search and LSS. To optimize search performance, the Agent Service maintains a memory cache of pats and other information. In addition, the Agent Service performs CPU-intensive operations like phonetic index creation, classification, and language

Type

Description

identification.

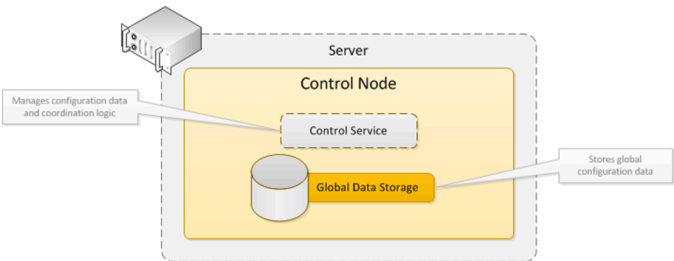


Figure 1. Control Node Components

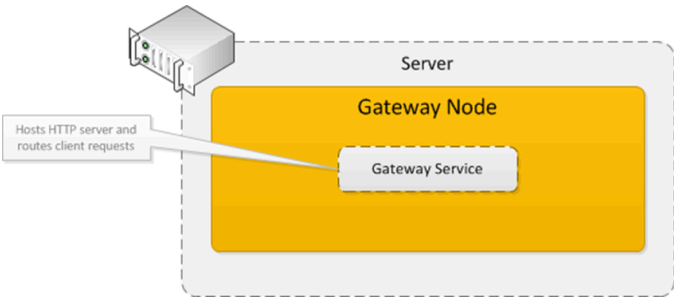


Figure 2. Gateway Node Components

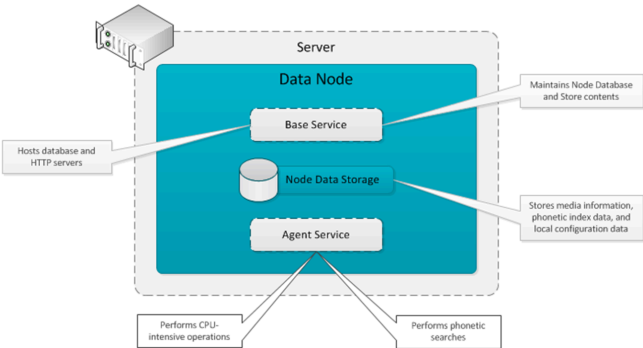


Figure 3. Data Node Components

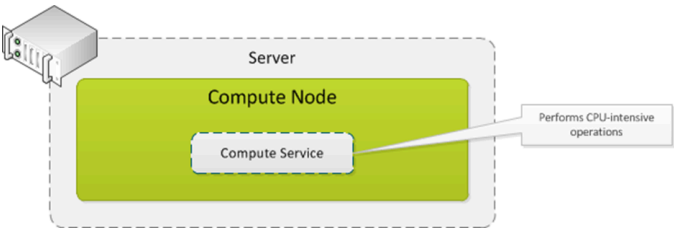


Figure 4. Compute Node Components

Common Deployment Configurations

Although many deployment configurations are possible, most variations in data volume and availability requirements can

be met with one of the following three options.

Single Server Deployment

In environments with a small data volume, a single server may suffice. The server hosts a Control Node, Gateway Node, and a Data Node. Because the Data Node can perform CPU-intensive operations, no Compute Node is needed.

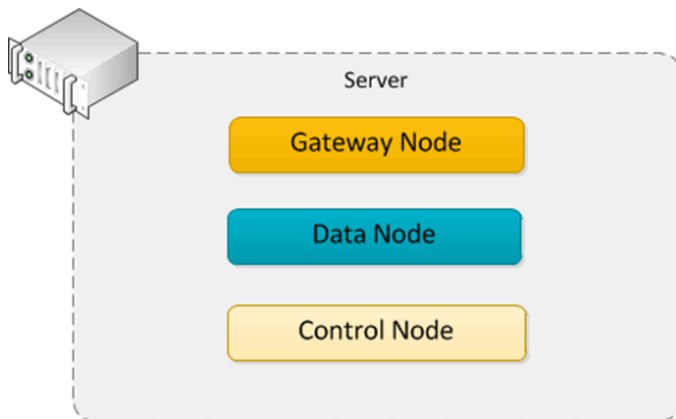


Figure 5. Single Server Deployment

Small Distributed Deployment

If the data volume is large enough to require two or three Data Nodes, the Gateway and Control Nodes should be installed to a separate server. This prevents IO and CPU load from the Data nodes from interfering with the Control Node; however, because there are no redundant single Gateway or Control Nodes, their server is a single point of failure.

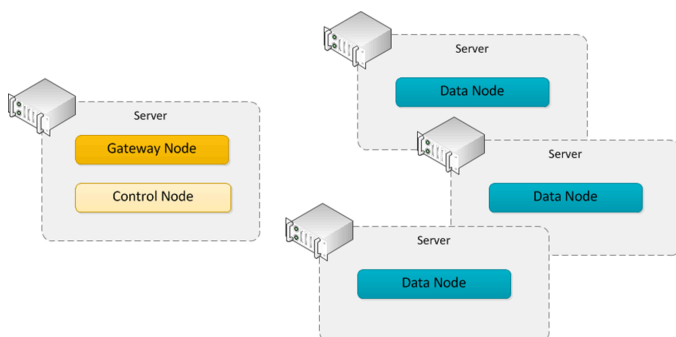


Figure 6. Small Distributed Deployment

Large Distributed Deployment

If the Data Node count is larger than three (or if high availability is required), it is recommended to host Gateway and Control Nodes on three separate servers. For failure isolation, it is crucial that the three Gateway/Control servers not share hardware, e.g. network storage, or a VM host.



Expansion of an *existing* deployment from one Control Node to multiple Control Nodes is possible, but requires planning and following instructions carefully, with validation checkpoints, in order to prevent data

loss. Therefore, please read and fully understand the [Adding or Removing Gateway and Control Nodes](#) section prior to beginning any such endeavor, and do not hesitate to consult with NiCE R&D first.

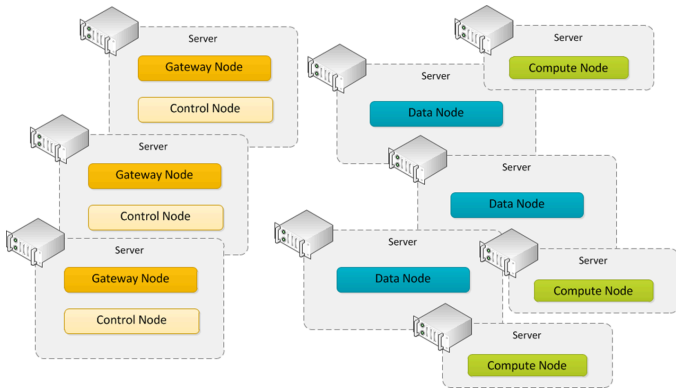


Figure 7. Large Distributed Deployment

A system with numerous Data Nodes should typically be used in *high availability mode*, which allows media declarations as long as at least one Data Node is up. For more information about the benefits and other important considerations of high availability mode, see the [Media Declaration Availability section of the Programmer's Guide](#). For information about enabling the media declaration high availability mode of Data Nodes, refer to its global property in the [Configuration Property Reference](#) section of this document.

Logical Networks

Search Grid communicates over three distinct logical networks:

Type	Description
Public	The public network is used for communication between client applications and grid nodes. The Gateway Node and Data Nodes communicate over the public network.
Grid-Private	The grid-private network supports communication between grid nodes. All node types need access to the grid-private network.
Node-Private	The node-private network is used for communication between services within a single Data Node.

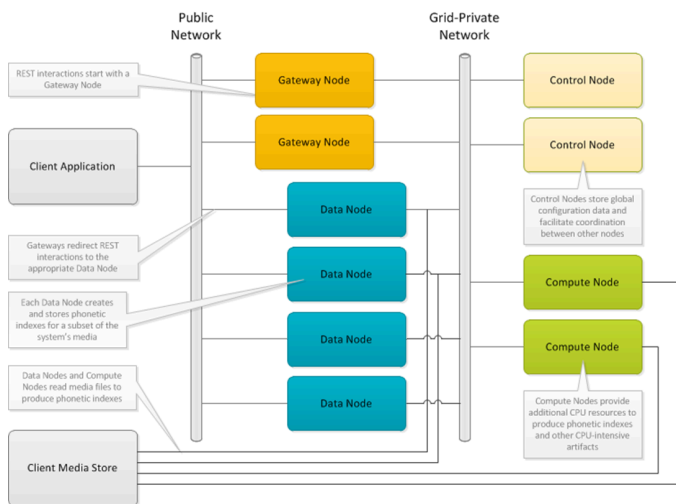


Figure 8. Logical Networks

The logical networks can be assigned to separate network interface cards or combined on a single physical network. If defaults are selected during the installation process, all logical networks are assigned to the loopback interface (127.0.0.1) by default.

Under most circumstances, the public network should be reassigned to a physical network interface card (NIC) so that client applications can interact with Search Grid from remote servers. The grid-private network can share the same physical network interface as public, but assigning it to a separate interface is recommended for two reasons. First, it isolates inter-node network traffic from the public network. Second, it increases security because the grid-private network can be made inaccessible from the public network. The node-private network binding to 127.0.0.1 should not be changed.

If the public network cannot be physically isolated from the grid-private and node-private networks, then the ports used in the latter two networks should be secured by other means, such as a firewall or IPsec. See the [TCP Port Usage](#) section for a list of default ports on each logical network.

For increased security, the public network can be configured to use HTTPS rather than HTTP. Use of HTTPS is recommended if the public network is untrusted to protect credentials and other sensitive user data. See [HTTPS Support](#) section for more information.

Each logical network can be bound to either an IPv4 or IPv6 address, but not both. Binding to an IPv4 address will prevent IPv6 connections and vice versa.

Installation

Prerequisites and Recommendations

See System Requirements document for compatible hardware and operating systems.

Due to Search Grid's IO-intensive nature, it is recommended to disable all virus and malware scanning software. In particular, Malwarebytes Anti-Malware, Kasperky Endpoint Security, and Microsoft Security Essentials are known to interfere with media operations such as pat creation. At a minimum, the following locations should be excluded from scanning:

- The Search Grid install location
 - usually `C:\Program Files\Nexidia\Search Grid X.Y` (where X.Y is the major.minor version, such as 20.1)
- The Search Grid data location
 - usually `C:\ProgramData\Nexidia\Search Grid`
- Folders where media is stored.

GPU Acceleration

Some Search Grid operations can benefit from GPU acceleration. Several factors affect whether and how much GPUs can help, so it is important to evaluate the application's use cases to decide whether GPUs are worth the investment.

- *What operations are in use, and in what proportion?* Two operations, phrase recognition and pat creation, can benefit from GPUs in some circumstances. A GPU increases phrase recognition performance by anywhere from 30% to 900% depending on the version of phrase packs being used, though the speedup varies widely across different media based on speech content and duration. A GPU roughly doubles pat creation performance (but see the next point).
- *What language pack or phrase pack versions are in use?* Language pack and phrase pack versions affect GPU acceleration in two ways: whether a GPU can help at all, and what kind of GPU is required. Only language packs version 8.7 or greater benefit from a GPU during pat creation, and only phrase packs version 8.7 and greater benefit from a GPU during phrase recognition. The greatest speed up is for the version 10.8 small (about 3x) and large transformer models (8-10x). Newer language and phrase pack versions may need more recent GPU models; see the System Requirements document for complete compatibility information.

Pre-Installation

See System Requirements document for compatible language packs and storage requirements.

- Decide how many nodes to deploy:
 - one or three Control Nodes
 - one, two, or three Gateway Nodes

- at least one Data Node
- For simplest administration, all Data Nodes should have similar CPU resources, IO performance, and storage resources.
- Establish the node name for each Data Node to be deployed. Note that node name is logically distinct from host name. Using the host name as the node name is not recommended because it would make it confusing to migrate the Data Node to a different server later. For more information, see the [Configuration Property Reference](#) section on "nexidia.searchgrid.node.name".
- Establish the control connection string to be used. This is a comma-delimited list of grid-private IP addresses of servers hosting Control Nodes. All servers must use the same value for this property, which is why it is important to determine the value before running the installers in a distributed deployment. For details, see the [Configuration Property Reference](#) section on "nexidia.searchgrid.control.connectionString".
- Configure any firewall software to allow incoming TCP connections on the ports listed in the [TCP Port Usage](#) section.
- If your organization prohibits the use of the bundled JRE, install an Oracle Java Runtime Environment (JRE) meeting the version requirements stated in the NiCE Search Grid Systems Requirements document.
 - This option is intended only for compliance with policies at specific deployment sites. In general, it is discouraged because the resulting configuration will not have undergone testing.
 - Only Oracle Server JRE version 8 is supported. The non-server distribution with browser integration and automatic updates is not supported.
 - Using the Control Panel, create a system environment variable NXGRID_JAVA_HOME and set its value to the location of JRE. The JRE location should have a bin/server directory with a jvm.dll in it.

Installation Procedure

The following steps describe a fresh installation on a server without a pre-existing Search Grid instance. For upgrade installations, see the [Upgrades](#) section.

Installation Steps

1. On each server to host a grid node, run the installer by double-clicking on NiCESearchGrid-<version>.msi.
2. On the "Choose Setup Type" step, click the button representing the type of installation desired for this server. For non-distributed deployments, choose the "Single Server" option. To install a combination of nodes not shown on the dialog, select one of the nodes you want and proceed to the next step, where you will have the opportunity to customize the components.

3. On the "Custom Setup" step, change the install and data locations if desired. Also, customize the node types to install if the "Choose Setup Type" step did not offer the desired combination. If you have installed a JRE and set the NXGRID_JAVA_HOME variable, the JRE option will not be selected. You can elect to install the bundled JRE but Search Grid will use the JRE defined by the NXGRID_JAVA_HOME variable.
4. On the "Configure Service" step, set the credentials under which the services will run. The default, NetworkService, is often sufficient for single server deployments, but see the [Service User Accounts and Permissions](#) section for details. Note that if the service user account is changed after installation, file permissions on the install and data locations may need to be updated.
5. On the "Ready to install NiCE Search Grid" step, click the "Install" button and accept the UAC prompt that follows. Subsequent steps collect information for the initial configuration settings.
6. Values chosen on the "Configure Local Properties" step will affect properties in the `<install location>\etc\local-properties.xml` file.
 - A. Set the bind addresses for the public and grid-private networks according to the guidelines in the Logical Networks section.
 - B. Set the control connection string to the value determined during pre-installation. Remember that all servers in the Search Grid instance must be set the same value.
7. Values chosen on the "Configure Node Properties" step will affect properties in the relevant node-specific configuration file. Some settings may not appear in the dialog depending on what nodes were chosen for installation.
 - A. If a Control Node is being installed, the Data Node Name list can be set in this step. As discussed in the Pre-Installation section, list all Data Nodes planned for the deployment by name. The list is comma separated and case-sensitive. The value will be transferred to the global properties the first time the Control Service is started.
 - If multiple Control Nodes are being deployed, this value needs to be set on only one of them; the value will propagate across all the Control Nodes after they are started.
 - See the [Global Properties](#) section on `nexidia.searchgrid.nodeNameList` for more details.
 - B. If a Data Node is being installed, the node name can be set in this step. The install process will update the corresponding property in the `<install location>\etc\data-node\data-node-properties.xml` file.



If you are adding one or more new Data Nodes to a live Search Grid, after installation is complete on the new node(s), the new node name(s) must be added to the global property `nexidia.searchgrid.nodeNameList` (see the [Global Properties](#) section), then all services on all nodes must be restarted.

Upgrades

When performing an upgrade installation, Search Grid can migrate settings from previous versions. Because uninstalling Search Grid does not remove old configuration files, settings can be migrated even if the previous Search Grid version has been uninstalled.



Search Grid 12.4 and later does not directly support upgrading from anything less than Search Grid 3.1. To upgrade from 1.1 or 2.0 to Search Grid 12.4 or later will require first upgrading to Search Grid 3.1 and following Search Grid 3.1's Administration Guide to do that upgrade.

The first time services start after an upgrade, Base Services apply any format changes to persisted data. This procedure can take minutes or hours to complete, during which Search Grid will be inaccessible to client applications. Apply this rule of thumb to estimate how long the update will take:

$$\text{expectedTimeHours} = \frac{\text{totalMediaCount}}{5 \text{ million} * \text{dataNodeCount}}$$

So on a system with 11 million media and four Data Nodes, it would take about 33 minutes for the update to complete.

During an upgrade, there are some restrictions on what cannot be changed in the Search Grid installer GUI.

Prior to 3.1.29, edits were allowed in the installer GUI, but some were ignored, which could have caused confusion, while other changes could cause problems with Search Grid services running. As of Search Grid 3.1.29, the Search Grid installer GUI restricts those items from being changed:

- Install Location field populates with the previous installation's location and cannot be edited *unless* the upgrade is an upgrade of X or Y for a version of X.Y.Z. (In other words, if the upgrade changes only Z in X.Y.Z — sometimes referred to as a "service pack upgrade" — then the Install Location cannot be edited.)
- Data Location field populates with the current data location and cannot be edited.



- Service User Name & Service User Password:
 - If 3.1.29 or later had *ever* been installed, the Service User field will default to the values used at the most recent successful installation or upgrade of 3.1.29 or later and cannot be edited; **however**, it is *essential* to fill in the *correct* password for that user, or it will cause the Search Grid Services to fail.
 - If 3.1.29 or later had *never* been installed, the Service User will default to "NT AUTHORITY\NETWORK SERVICE" with its password; **however**, it is *essential* to change these fields to whatever the current Service User name and password is. Failure to do so will result in errors with the Search Grid services.

If you have uninstalled the previous version of Search Grid, as mentioned above, all data at the prior Data Location is retained and all configuration properties in the Installation Location are retained in the

`<install location>\etc` directory. By default, the installer GUI will populate the Install Location field with the default location for the version being installed and *will* be editable, and the installer GUI will populate the Data Location and Service User fields as described in the bulleted items above, *except* that they *will* be editable. This is to facilitate changing to a new Install Location and/or Data Location if previously installed to location in error or if the intent is to start out with fresh locations and/or new configuration properties, which may be more likely in a lab environment. **However:**

- *If you want to retain the data from a previous installation*, make sure you set the Data Location to that previous location.
- *If you want to retain the configuration properties from the previous installation*, make sure you set the Install Location to that prior installation's directory, or after installing, copy everything from the prior installation's 'etc' directory to your new one.
- If you re-use a prior Install Location, *and it contains configuration properties in its etc subdirectory*, the Data Location will **ignore what was set in the installer GUI**, and instead will be the Data Location as specified in those properties files. (This may seem unintuitive, but it safeguards against data loss, and if someone had uninstalled with the intent to re-install *completely fresh*, it is their responsibility to use a new Install Location or have deleted the previous Install Location to guarantee that it is a truly fresh installation.)

Upgrade Steps

Stop all Search Grid services.

1. Close the Windows Services GUI, and if it is open, also close the Windows Event Viewer GUI. (This is very important because either being open can cause upgrade failures under certain conditions.)
2. Back up the contents of all data locations.
3. Run the installer. If the old version of Search Grid is currently installed, the installer will automatically choose the same set of features for the new version.
4. The installer will prompt for the previous version's install location to migrate settings. If the old version was installed to a non-standard location, you may need to change the directory from which to migrate.
5. Make any manual configuration changes mentioned in the README file.
6. Start the services. The first time services start after an upgrade, they will apply any format changes to persisted data. This procedure can take several minutes to complete, during which Search Grid will be inaccessible to client applications.



If after an upgrade, services fail to start, do not only check that service's log files for errors, but also check the Windows Event Viewer for logged application events from the failing services. (Event Viewer → Windows Logs → Application).

Post-Installation Procedure

Start the Search Grid services. Services may be started in any order, but the order shown below will result in the fastest startup time.

Node Type	Recommended Start Order
Control Node	• NiCE Search Grid Control Service
Data Node	• NiCE Search Grid Base Service • NiCE Search Grid Agent Service
Gateway Node	• NiCE Search Grid Gateway Service
Compute Node	• NiCE Search Grid Compute Service

Clients may now connect via the System URI based on the value of `nexidia.searchgrid.this.public.bindAddress` in the `local-properties.xml` file of *any* Gateway Node. The Java and .NET client libraries support configuration with multiple System URIs, if multiple Gateway Nodes are available.

■ `http://<nexidia.searchgrid.this.public.bindAddress>:25002/`

Note that if HTTPS support is enabled, the System URI starts with "https" instead of "http".

Service User Accounts and Permissions

All Nodes

By default, the installer registers the Windows services to run under the "Network Service" account. If this account is changed *after* installation, the new service user needs read permission to the entire grid installation folder tree, usually `C:\Program Files\Nexidia\Search Grid X.Y` (where X.Y is the major.minor version, such as 20.1).

Also, the service user *requires* "Full Control" permission to the node's data locations, which are all subfolders of the <data location> set at installation. (The default <data location> is `C:\ProgramData\Nexidia\Search Grid`, though usually the default is not used, and a path on a drive other than `C:` is set at installation, often <some other drive>:\ProgramData\Nexidia\Search Grid.) If configuration files are edited post-installation to use non-default paths, appropriate permissions must be applied to the new locations and Search Grid's registry values should be edited to reflect the change. See the [Moving Search Grid's Folders](#) section.



It is highly recommended that the service user be a member of the local **"Performance Logs Users"** group on each server where Search Grid is running. This allows Search Grid to directly report CPU utilization information, which is extremely helpful to have if there is ever an issue requiring support. (More specifically, the service user needs the `"SeSystemProfilePrivilege"` privilege, which is granted by adding the service user to the **"Performance Log Users"** group.)

+ If the management-console's `test-system -verbose` command reports "Unavailable" for CPU utilization, then the service user is missing that permission and needs to be added to that group.

Data Nodes and Compute Nodes

Data Nodes and Compute Nodes read media files to create pats, so the Agent Service and Compute Service user needs appropriate read permissions for the media files. The Agent Service and Compute Service may be assigned a different account than the other Search Grid services so long as the account also meets the criteria described in the [All Nodes](#) section above.



A Compute Node should never be installed on the same server as another node type; they're intended to run on their own servers, with freedom to consume the full resources of the server. Running a Compute Node on a server with other node types, especially Data Nodes, will not just negatively impact the performance of the other node type's services; it will cause over-allocation of RAM, likely causing service and possibly even operating system crashes. Careful customization of the phrase recognition properties on that server may avoid RAM over-allocations and crashes, but the Compute Service may still negatively impact the performance of the other services, and therefore is not a supported Search Grid configuration.

Installer Command Line Options

The installer supports several command line options intended to facilitate automated installation, e.g. from a script.

- Use `msiexec /i NiCESearchGrid-<version>.msi` with a set of features to invoke the installer to install.
- Use `msiexec /x NiCESearchGrid-<version>.msi` to invoke the installer to uninstall.
- In automated installation scenarios, the `/qn` option should be specified to invoke silent installation.

Enabling features:

If no features are selected, only Search Grid applications—and no services—will be installed. This is rarely useful, so in most situations, additional features should be selected. This can be done either by explicitly listing features in the `ADDLOCAL` property, or by using the `SINGLE_SERVER_INSTALL` property.

To select all features necessary for a single-server deployment, set the `SINGLE_SERVER_INSTALL` property to "true" like so:

```
msiexec /qn /i NiCESearchGrid-<version>.msi SINGLE_SERVER_INSTALL=true
```

To get more fine-grained control over the installed features, specify them in the `ADDLOCAL` property as a comma-delimited list. Valid feature names are as follows:

- JRE (required unless custom JRE is in use)
- ControlNode
- GatewayNode
- DataNode
- ComputeNode

Other options:

The following properties can be set to change where grid is installed:

- `INSTALLDIR`
 - On new installations and on upgrades where X or Y changed in version X.Y.Z, this defaults to `[ProgramFiles64Folder]\Nexidia\Search Grid X.Y\`
 - On micro version upgrades, where only Z changed in version X.Y.Z, this defaults to the current installation location.
- `DATA_INSTALLDIR`
 - On new installations, this defaults to `[CommonAppDataFolder]\Nexidia\Search Grid\`
 - On all upgrades, this defaults to the current data location.

The values of `CommonAppDataFolder` and `ProgramFiles64Folder` depend on the version of Windows. See Microsoft Installer documentation for complete explanation.

Use the following properties to customize the service user account:

- `SERVICEUSER=<username>`
- `SERVICEUSERPASSWORD=<password>`

Upgrade installations:

The following properties can be set to migrate settings from a previous Search Grid installation:

- MIGRATE_SETTINGS=1 (to enable setting migration)
- PRIOR_INSTALLDIR=<previous install location, usually C:\Program Files\Nexidia\Search Grid X.X\>

Examples

Typical single-server installation:

```
msiexec /qn /i NiCESearchGrid-<version>.msi SINGLE_SERVER_INSTALLATION=true
```

Minimal (apps-only) installation:

```
msiexec /qn /i NiCESearchGrid-<version>.msi
```

Install Data Node using the M drive for data storage:

```
msiexec /qn /i NiCESearchGrid-<version>.msi ADDLOCAL=JRE,DataNode DATA_INSTALLDIR=M:\gridstorage\
```

Single server installation without bundled JRE:

```
msiexec /qn /i NiCESearchGrid-<version>.msi ADDLOCAL=ControlNode,GatewayNode,DataNode
```

Uninstallation:

```
msiexec /qn /x NiCESearchGrid-<version>.msi
```

Tips:

- Must run from an elevated command prompt to use /qn.
- msiexec returns before installation/uninstallation is complete.
- If using /qn, any installation errors will cause the installation to fail silently.
- Paths must include a trailing backslash.
- Paths with spaces must be in double quotes.
- Cannot install if another version of Search Grid is already installed.
- Uninstalling does not remove configuration files or data files (pats, polygests, database, etc).
- Reinstalling (i.e. uninstalling, then installing the same version) does not overwrite configuration files or data files.

Control Node Inbox

The Control Node inbox is a directory (`<data location>\control-node\inbox`) that can be used to automatically inject global configuration properties into the Control Node ensemble. Unlike the Management Console technique, which can also be scripted, the Control Service does not have to be running in order to submit input data when using the inbox technique. Therefore, the inbox can be especially helpful in automated deployment scenarios.



If the Control Node(s) can be running, the Management Console scripting technique is simpler and preferred. If not, the inbox technique is necessary.

As a Control Service starts, it checks the contents of its inbox directory. If a recognized filename is present, the Control Service writes its contents to the ensemble as soon as a quorum is available. Once the write has been completed successfully, the Control Service deletes the input file from the inbox. If the write fails or the input file is invalid, the Control Service terminates.

The inbox supports automated installation scenarios in which a script or some other tool is invoking the installer (msiexec on Windows). The installer creates the inbox (`<data location>\control-node\inbox`), which is readable and writable by the service user account. The script may write the relevant files into the inbox directory, ensuring that the service user account has read and write permissions for each file.

In a deployment using multiple Control Nodes, only one node needs to get the data, which it will then distribute to the other Control Nodes; however, the script *must* ensure there are no conflicting data files in the inboxes of *every* Control Node. Therefore, the script should write the same data files into each node's inbox OR write the data files to one node's inbox, and remove any files from the other nodes' inboxes.



Control Services do not monitor their inbox during normal operations. The inbox is checked only during service startup.

- In the case of the script writing to only one Control Node's inbox, if running, that node's Control Service **MUST** be restarted.
- In the case of multiple Control Nodes, if the script wrote to all nodes' inboxes, if running, ALL nodes' Control Services **MUST** be restarted as close to simultaneously as practical.
- In all cases, *all other Search Grid services* (on all nodes) **must** be (re)started *after* (re)starting the Control Service(s) in order for those services to see the changes.

For more details about specific inputs to the inbox, see the [Global Configuration Management](#) section of this document.

Post-Installation

Once all nodes are installed and the services started, Search Grid requires some administrative steps before the application is fully functional.

- At least one language pack needs to be installed. See the [Language Pack Management](#) section for more

information.

- If Search Grid's multi-tenancy support is to be used, additional tenants should be created. See the [Tenant and Credential Management](#) section for more information.

Administration

Management Console

Grid includes a Management Console application that allows users to create and delete tenants, change passwords, and other useful functions. The Management Console is installed as part of the "Apps" feature. Run the application by double-clicking the following file in the install location:

```
| <install location>\bin\management-console.cmd
```

On startup, the application will prompt to enter the user and the user's password. Depending on the credentials entered, the application will enter one of three modes:

- *Admin Mode* supports the largest number of management commands, but does not allow direct access to tenant data. To enter Admin Mode, enter user "admin" and the admin user's password. (The default admin password is "admin".) After logging in, the prompt will be a hash symbol, '# '.
- *Tenant Mode* allows the user to inspect and manipulate phonetic data. (By default, a tenant's password is the same as the tenant name. The name of the tenant available in new installations is "DefaultTenant".) Once a tenant user is logged in, the prompt will be a right angle bracket, '>'.
- *Primitive Mode* is the available mode if no Gateway Nodes or less than a majority of Data Nodes are running. Primitive Mode supports a restricted set of commands for managing global configuration data and credentials. To enter Primitive Mode, simply press return at the user prompt; the command line prompt will be an exclamation point, '!'.

Subsequent sections of this document describe specific Management Console commands and functions. You can also enter "get-help" at the console prompt for a summary of available commands, including parameters and brief descriptions. (The command list will vary depending on which of the three aforementioned modes you are in.)



Using "get-help -brief" will list only the command names and aliases without parameters or descriptions.

There are also some "hidden" management console commands that apply to each of the three aforementioned modes. They are hidden from the normal get-help listing because they are not commands that would normally be used for Search Grid operation; however, they might be useful for NiCE personnel to troubleshoot some support issues. Using "get-help -hidden" will list all (and only) the available hidden commands for the current management-console mode.

Management Console Scripting

In addition to interactive use, Management Console also offers simple scripting support. Using scripting support, Management Console functions can be accessed by administration tools such as system scripts (e.g. CMD and PowerShell) and the Windows Task Scheduler.

When invoking Management Console interactively, it prompts the user for credentials; however, interactive credential entry is not available in a scripting context, so two alternative means of credential entry are available. One way is to specify the user and password on the command line using the `-u` and `-p` options, respectively. Because the `-u` and `-p` options expose credentials in the command line invocation, access to the calling script should be tightly controlled. The other alternative is to set credentials via the following settings in `<install location>\etc\apps\apps-properties.xml`:

```
<entry key="nexidia.searchgrid.management.console.user"></entry>
<entry key="nexidia.searchgrid.management.console.password"></entry>
```

Note that the config file is installed as world-readable, so if credentials are placed in the config file, its read permissions should be restricted.

Scripted Management Console commands may be specified in two different modes. The first, *immediate mode*, is a simple way to run a single command. The second, *command file mode*, involves passing Management Console a separate file containing a set of user inputs.

Both immediate mode and command file mode are affected by the `-f` option ("force"). When `-f` is specified, interactive prompts such as language pack selection and delete confirmation are suppressed, and default responses are assumed. The `-f` option is required for most commands in immediate mode. The `-f` can simplify use of command file mode because command files do not need to include user responses to prompts.

To use immediate mode, specify the command and its arguments after any options to Management Console itself. For example:

```
management-console -u admin -p mypassword -f add-languagepack "E:\LanguagePacks\
NAEnglishTeleUniversal-8.0.1.91034"
```

Management Console also accepts a *command file* containing multiple commands, one command per line. Specify the command file via the `-c` option. For example:

```
management-console -u admin -p mypassword -c my-command-file
```

Command files must contain a literal transcript of what a console user would enter. This includes any confirmations to prompts unless the `-f` option is specified.

Calling scripts can use Management Console's exit code to determine whether the commands succeeded. If all commands succeed, Management Console exits with exit code 0. If a command fails, Management Console exits with exit code 1. If Management Console was invoked with a command file, command processing terminates at the first command to fail. Specific information about the error will be available in the apps log file, usually in [node data

location]\logs.

Tenant and Credential Management

As discussed in the Programmer's Guide, most Search Grid operations require a set of credentials; that is, a user and a password. When installed, Search Grid has two sets of predefined credentials:

- Administrator credentials, user and password both set to "admin"
- Tenant credentials for the predefined tenant, user and password both set to "DefaultTenant".

Management Console Commands

Except for set-password, the following commands are available only to the admin user.

- To change the default admin password:

```
# set-password admin
```



If the system is configured for Disaster Recovery the admin password must be the same on the primary and the backup.

-
- To list the currently defined tenants:

```
# get-tenant
```

- To get information about a specific tenant:

```
# get-tenant TenantName
```

- To create a new tenant named "MyNewTenant"

```
# create-tenant MyNewTenant
```



This command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.

-
- To change the password for the new tenant (as admin user)

```
# set-password MyNewTenant
```

- To change the password for the new tenant (as tenant user)

```
| > set-password
```



This command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.

- To delete the predefined tenant

```
| # delete-tenant DefaultTenant
```



This command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.

Note that the licence must enable the multi-tenant feature to support more than one tenant at a time. Otherwise, any existing tenant must be deleted prior to creating a new tenant.

When invoking delete-tenant in immediate mode, the -f option should always be specified to suppress the confirmation prompt.

Administrator User

The administrator user name, "admin", cannot be changed. Changing the admin password, however, can be done from the Management Console by logging into the Management Console as admin using the current password, then using the set-password command as shown above. Changing this password from the default password is recommended to improve security.

If the admin password has been lost, the admin password can be set to a new one; however, for security reasons, this will require contacting support for assistance.

User Lockout

To prevent brute-force password guessing, Search Grid locks users out based on a pattern of authentication failures. Both admin and tenant users are subject to lockout. While a user is locked out, no API functions can be invoked under the user's credentials. However, operations like pat creation and classification will continue to execute if they were requested prior to the lockout.

Search Grid locks a user out when a specific number of authentication failures, the *lockout threshold*, occur within a specific length of time, the *lockout window*. The user remains locked out until one of the following criteria are met:

- A period of time, called the *lockout duration*, passes.
- The administrator unlocks the user's account via Management Console.
- The administrator unlocks the user's account programmatically via a client application.
- The Gateway Service is restarted.

Note that the administrator user's account can be unlocked only by using Management Console in primitive mode. For security reasons, the public API cannot be used for this purpose.

The configuration properties governing the lockout policy are detailed in the Global Properties section of the Configuration Property Reference.

Management Console Commands

- To list users that have been locked out (as admin user only):

```
# unlock-user
```

- To unlock a tenant user that has been locked out (as admin user only):

```
# unlock-user tenantname
```

- To unlock the administrator user (in primitive mode only):

```
! unlock-user admin
```

Language Pack Management

Search Grid requires at least one language pack to perform phonetic analysis. Choosing language packs can be a complex decision that should involve consulting with NiCE technical support. See the System Requirements document for compatible language pack versions.

Language packs are distributed as .msi files. These packs are installed to a directory on the server (typically `C:\Program Files (x86)\Nexidia`) and are then installed to Search Grid. Once the language pack has been installed to Search Grid, it can be uninstalled from the server via the Programs and Features control panel. Installing a language pack to Search Grid can be accomplished through Management Console, by a client application, or automatically. Subsequent sections describe each of the three options.

Once a language pack is installed to Search Grid, it can be uninstalled from the server; however, language packs cannot be uninstalled from Search Grid itself.

Legacy Model Packs

Language packs in the 7.x series did not include search models or phonetic digest models. Instead, search models and

phonetic digest models were distributed in separate *model packs* installed independently of the corresponding language pack. These legacy model packs can be installed into Search Grid to enhance the functionality of the corresponding language pack.

When searching a 7.x language pack without a search model pack installed, the `applySearchModel` option will have no effect. The search will complete without errors, although Gateway Service log will contain a message stating that a search model is unavailable.

Additionally, a 7.x language pack without a phonetic digest model pack will not support large scale search. Attempts to use large scale search against the 7.x language pack will result in an error. If a phonetic model pack is installed to a Search Grid instance with pre-existing pat files, Search Grid will create digests for them. Depending on the volume of pre-existing pats, digest creation could take hours or days.

Automatic Language Pack Installation

The Gateway Service can periodically scan a directory for language packs and automatically install any language packs it finds. To enable this feature, add the following line to `<install location>\etc\gateway-node\gateway-service-properties.xml`:

```
<entry key="nexidia.searchgrid.languageservice.searchPath">C:\Program Files (x86)\Nexidia</entry>
```

The "C:\Program Files (x86)\Nexidia" directory is the default install location for language packs on most systems, but you may need to enter a different value on your server. Multiple directories can be specified by separating them with a semicolon. Disable automatic language pack installation by removing the property or setting it to a blank value.

Automatic language pack installation is convenient, but it also has several drawbacks:

- All discovered language packs are installed. This can be problematic in a server to which several language packs have been installed, but only a subset are intended for use with Search Grid.
- It can be difficult to know precisely when the new language pack is available in Search Grid. Administrators can use Management Console or monitor the Gateway Service log. Client applications can invoke the Get Language Pack Inventory operation.
- Scanning for new language packs consumes a small amount of disk and CPU resources on the Gateway Node.
- It does not support language packs in archive files.

Legacy Model Pack Support:

Automatic language pack installation identifies and installs legacy model packs as well as language packs.

Language Pack Installation via Management Console

Management Console offers several commands that facilitate language pack management, all of which require admin credentials:

- To list language packs already installed to Search Grid:

```
| # get-languagepack
```

For 7.x language packs, the presence of legacy model packs is indicated in the description of the corresponding language pack. Legacy model packs installed prior to the corresponding language pack are not displayed.

- To list language packs in a directory available for installation to Search Grid:

```
| # get-languagepack <directory>
```

Management Console presents a numbered list of the packs with the path of each pack's subdirectory. Legacy model packs are displayed as well. The directory must be readable by the Management Console application, but does not need to be read by Search Grid services.

- To install a language pack to Search Grid from a directory:

```
| # add-languagepack <directory>
```

Management Console will display language packs and legacy model packs as a numbered list and prompt for which pack to install. The directory must be readable by the Management Console application, but does not need to be read by Search Grid services.

- To install a language pack to Search Grid from an archive file:

```
| # add-languagepack <file>
```

If you attempt to install a language pack that has already been installed to Search Grid, Management Console will report a conflict and no changes will be made.



The `add-languagepack` command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.



The `-f` option must be used to run the `add-languagepack` command in immediate mode; otherwise, the command will fail. (This is because the user is always prompted for language pack selection.) If the specified directory contains only a single language pack, that pack will be installed. If the specified directory has multiple language packs, Management Console will exit with an error.

Programmatic Language Pack Installation

Client applications can use the Add Language Pack and Add Legacy Model Pack operations to install packs via the Search Grid API. See the Programmer's Guide for more information. If programmatic installation is used, the language pack must be installed on a server accessible to the client application, and does not have to be directly accessible to Search Grid components.

Phrase Pack Management

To perform phrase recognition, a tenant must have at least one phrase pack installed. Techniques for managing phrase packs are almost identical to those for managing language packs, with a few important differences:

- Phrase packs are tenant-specific resources, not global resources like language packs.
- Phrase packs can be removed from Search Grid after installation.
- Phrase packs cannot be installed automatically.

Phrase packs are usually distributed as archive files, either .tgz or .zip. These files can be extracted to a server, but it is more efficient to install them directly to Search Grid. On either platform, installing a phrase pack to Search Grid can be accomplished through Management Console or by a client application. Subsequent sections describe each option.

Phrase Pack Installation via Management Console

Management Console offers several commands that facilitate phrase pack management, all of which require tenant credentials. NOTE: A common mistake is to attempt to manage phrase packs using admin credentials; none of these commands are available when logged in as an administrator.

- To list phrase packs already installed for this tenant:

```
| > get-phrasepack
```

- To list phrase packs in a directory available for installation to Search Grid:

```
| > get-phrasepack <directory>
```

Management Console presents a numbered list of the packs with the path of each pack's subdirectory. The directory must be readable by the Management Console application, but does not need to be read by Search Grid services.

- To install a phrase pack to Search Grid from a directory:

```
| > add-phrasepack <directory>
```

Management Console will display phrase packs as a numbered list and prompt for which pack to install. The directory must be readable by the Management Console application, but does not need to be read by Search Grid services.

- To install a phrase pack to Search Grid from an archive file:

```
| > add-phrasepack <file>
```

If you attempt to install a phrase pack that has already been installed for the same tenant, Management Console will report a conflict and no changes will be made.



The add-phraspack command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.



The -f option must be used to run the add-phraspack command in immediate mode; otherwise, the command will fail. (This is because the user is always prompted for phrase pack selection.) If the specified directory contains only a single phrase pack, that pack will be installed. If the specified directory has multiple phrase packs, Management Console will exit with an error.

- To delete a phrase pack, provide the UUID for the phrase-pack you want to delete, as displayed by the get-phraspack command:

```
| > delete-phraspack <phraspack-uuid>
```



The delete-phraspack command will be disabled when Disaster Recovery is enabled and not paused. Refer to the warning in the [Configuring Disaster Recovery](#) section for an explanation of why.

Programmatic Phrase Pack Installation

Client applications can use the Add Phrase Pack operation to install packs via the Search Grid API. See the Programmer's Guide for more information. If programmatic installation is used, the phrase pack must be installed on a server accessible to the client application, and does not have to be directly accessible to Search Grid components. Remember that, because phrase packs are tenant-specific resources, the relevant management operations require tenant credentials rather than admin credentials.

System Tests and Diagnostics

Management Console's system tests provide a convenient way to diagnose the health of an entire distributed Search Grid system. The tests include exercise of communication paths, file system permissions, and other internal resources.

Administrators can run system tests at any time and in any system mode. However, the tests do incur a small amount of CPU, network, and file system load, so running tests during periods of low use may be preferred.

To examine system health without incurring significant load, use the "get-systemtopology" command.

Management Console Commands

All the commands below require admin user credentials.



The test-system, get-systemtopology, and get-gridMemoryInfo commands below report all data nodes, but for compute, gateway and control nodes, only up nodes are reported, therefore it is recommended to check the number of each node type reported by the get-systemtopology command against the number of actual nodes in order to determine if there are non-data nodes that are down.

- To run system tests and display their results:

```
| # test-system
```



The test-system command checks for an unacceptable time difference between nodes. By default, all nodes must be within +/- 5 seconds of the Gateway. Occasionally, if test-system is run shortly after start-up as things are still loading, or if there is very high load on a node or on the network between nodes, a time difference error could be reported due to that load creating a significant delay in response to node pings. If this occurs, simply run test-system again to re-test. If the error persists, verify and correct the time on the nodes that the issues are reported for.

- To run system tests and display each node's platform details (OS, CPU cores, RAM, etc.) as well as the test results:

```
| # test-system -verbose
```

- To run a system test, getting each node's detailed information but displaying only the detail lines that contain "CPU":

```
| # test-system -verbose:CPU
```



The string that follows the colon is case-sensitive and can be a regular expression (regex), but it *cannot* contain spaces. If it doesn't start with a "^" character (regex beginning of line) it is presumed the value can be at or after the beginning of a line (effectively, "." is prepended). If it doesn't end with a "\$" (regex end of line) it is presumed that the value can be before or at the end of a line (effectively, "." is appended").

- To view a list of nodes, their counts, detailed statuses, and summary:

```
| # get-systemtopology
```

- To view a list of node counts, plus detailed statuses of down nodes only, and summary:

```
| # get-systemtopology -brief
```

- To report CPU and memory usage information for every gateway, compute node, and data node in the grid:

```
| # get-gridResourceUtilization
```



This information can also be viewed in tabular format using a web browser and going to <http://<YourGatewayIP>:25002/about/gridResourceUtilization> and logging in with the admin password (same as management-console's).

- To report CPU and memory usage information for DataNode1:

```
# get-gridResourceUtilization DataNode1
```



The optional parameter to get-gridResourceUtilization is case-sensitive and can be a regular expression (regex), but it *cannot* contain spaces. If specified, only nodes with names matching the regular expression will be reported.

- To report the details of a single structured query, the details of all structured queries, or to get the number of structured queries stored:

```
# get-structuredquery (<structuredQueryId>|ALL|COUNT)
```



This command requires running the management-console with the user credentials of the **tenant** for which you want to view its structured queries.

Search Grid Metrics Commands

In addition to being able to query overall (grid-wide) metrics on demand using the commands below, the Gateway, Base, Agent and Compute services all periodically log their metrics to separate "performance log" files. (See [Log File Configuration](#) for more about those files.) This allows checking individual node performance as well as overall grid performance.



- All *except* the Gateway service's performance logging is that *specific* node's relevant metrics, and the logging is driven by the work being done on that node. By default, logging happens at most once every 15 minutes while work is being done, and no longer than one hour from completion of work if changes had not yet been logged. No logging is done if no work is being done.
- The Gateway's performance logging is the aggregated performance metrics of the entire grid (basically,

the equivalent of the below commands "get-cmet", "get-dmet", "get-smet", and when applicable, "get-drmet") polled and logged periodically. By default, logging will happen at every 15 minutes for whatever metrics have changed since last logged, and at least once per day regardless of change.

- To view status of persistent data storage, including searchable volume:

```
| # get-datametrics
```

- To view status of persistent data storage for a specific tenant:

```
| # get-datametrics <tenant name>
```

- To view status of CPU-intensive tasks across the grid:

```
| # get-computemetrics
```

- To view status of search-related tasks across the grid:

```
| # get-searchmetrics
```

- To view the status of Disaster Recovery replication, if applicable (see <<Disaster Recovery section for more info):

```
| # get-drmetrics
```

Administratively Canceling Media Operations

Sometimes a client application may submit numerous media operation requests that are not relevant. This may be because circumstances have changed and the requests are now obsolete, or maybe there was a bug or misconfiguration in the client. In these circumstances, it is usually not desirable to allow the irrelevant operations to complete, and the client application often lacks the ability to programmatically cancel its requests. Administrative cancellation allows an administrator to cancel all media operations for a given tenant (or set of tenants). So long as the system is configured for administrative cancellation, all pending and new requests will complete with the AdminCanceled error code.

Note that administratively canceled requests still generate notifications if the request enabled the notification option.

Administrative cancellation is controlled by setting a global property to a comma-delimited list of tenant names. Tenants named in the list will have their media operations canceled until the tenant is removed from the list. So the following command in Management Console would cancel all operations for MistakenTenant and BuggyTenant:

```
| # set-gprop nexidia.searchgrid.cancelTenantsMediaOps MistakenTenant,BuggyTenant
```

When sufficient operations have been canceled, the following command would end administrative cancellation:

HTTPS Support

All HTTPS configuration options are global in that they affect all tenants and all client applications interacting with Search Grid. Several options imply requirements for client application behavior and configuration, so it is important to clearly communicate the HTTPS usage policy to client application developers.

HTTPS usage policies can be viewed as multiple levels, each level increasing security at the cost of increased administration burden:

- Level 1: Enable HTTPS with Search Grid's built-in certificate.
- Level 2: Create and use custom server certificates.
- Level 3: Allow client applications to authenticate server certificates.

A Little About Certificates

A certificate is used not only to encrypt the data using its public key (readable only by a receiver that has the matching private key), but also as a means of authenticating that the server is in fact the server that the client is expecting. To provide that, the certificate is signed, and the client validates that signature, however all signatures are not created equal.

Signature chains are used to validate authenticity. For example, the certificate used on a server may have been issued by an organization that is not a trusted root authority, but that organization has been authenticated by another organization, and so on up the chain to a trusted root authority.

- **Self-Signed Certificates:** The lowest-level of authentication is a self-signed certificate, which is not issued by any authority. As its name implies, anyone can create a self-signed certificate, and thus it really provides no guarantee of the server's identity, however it does allow data to be exchanged in an encrypted format. If an organization is extremely confident of its network security and the inability for any unauthorized devices to appear on its network, a self-signed certificate may be sufficient, but most organizations will not be satisfied with a self-signed certificate. Search Grid itself provides a self-signed certificate, internally, though it is really intended only for demonstration, lab environments, or proof-of-concept purposes, such as verifying HTTPS functions with a known-to-be-good certificate. An organization could generate their own self-signed certificate and use that for Search Grid, but given that it adds no additional level of security, it's fairly pointless, other than possibly (insecurely) identifying each Search Grid server vs. the same internal certificate used on all Search Grid servers.
- **Internally-Signed Certificates:** The next level of authentication is similar to a self-signed certificate, but is signed within the organization, and keyed specifically to the server on which it is to be installed. That organization then typically installs another certificate of its own – the one it uses to sign its other certificates — on all of its internal computers as a trusted root certificate, even though it's not truly authenticatable through a chain back to a trusted

root authority. This type of internally-signed certificate is popular on servers that are only internally-accessible because the outside world does not need to authenticate them up to a trusted root authority, and internally-signed certificates are free, whereas getting a certificate signed by an external trusted authority has a cost associated with it. In this case, the internally-signed certificates for the individual servers authenticate up the chain (usually just one level) to the other certificate that the organization has placed on its computers as a trusted root certificate. Within the organization, client apps are satisfied with authentication. The level of security, however, depends upon how securely that other "trusted" certificate is managed by the organization. To acquire the server-specific signed certificate, some organizations may have an IT Security department that creates the certificates upon request, while others may have the requester create certificate and keypair locally as a certificate signing request (.csr file), and then sent to their IT Security department to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore.

- **Trusted Authority Signed Certificates:** The most secure level of authentication is a certificate that is signed by a trusted signing authority, issued specifically to the server on to which it will be installed. This certificate can be authenticated up the signature chain to a true trusted root authority. A certificate and keypair are created locally as a certificate signing request (.csr file), and then sent to a signing authority to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore. This signing process is not free, and can take time to get, but offer the highest level of authentication. To use this type of certificate for internal servers like Search Grid is uncommon except for those organizations who are extremely concerned with security.

A Little About Keystores

A keystore can contain many certificates and their private keys. Some organizations may put all certificates for their various servers into one keystore, while others may have a separate keystore per server. For the sake of simplification of the instructions below, we'll have a single certificate in the keystore, but that is not a requirement.

There are also multiple formats of keystores, such as JCEKS, JKS, PKCS12 and others, the most common being PKCS12. There are also means to convert between most keystore types, if necessary.

Search Grid itself is capable of reading JCEKS and PKCS12 keystores. Search Grid is also capable of using certificates that are stored securely within Window's internal, secure Certificate Repository, and to import the contents of a keystore into Windows, a PKCS12 keystore is required.

Configuring Search Grid for HTTPS

Search Grid can be configured to use HTTPS (SSL encryption) on its public endpoints. The public endpoints are not only the Gateway/Control Node(s), but also all Data Nodes, as some APIs made by clients get redirected by the Gateway to the Data nodes. Almost all media-specific calls are redirected to the appropriate Data Node that contains that media. Audio segments from running classifiers, for example, are downloaded directly from Data Nodes. Therefore, to enable HTTPS encryption on all of that public traffic, **all Data Nodes must be enabled for HTTPS**, and if using custom certificates, each Data Node will need its own unique certificate.

After changing any of the properties in this section, you must restart all Search Grid services on those nodes in order for the changes to take effect.



If you are using the Disaster Recovery feature and configure the Backup instance to use HTTPS, you will need to specify **https://** in the `backupInstanceSystemUri` property. See the [Primary Instance Properties](#) and [Client Authentication of Servers](#) sections.

Default Certificate

As mentioned in the Self-Signed Certificate description above, Search Grid includes a built-in self-signed certificate that is used by default if no other certificate is specified. The built-in certificate is convenient, but it is *highly discouraged* from being used in a production environment for several reasons:

- The certificate is self-signed and thus does not feature a chain of trust to a well-known certificate authority.
- The certificate's Common Name will not match the server name, so client applications will be unable to verify server identity.
- All Search Grid installations contain the same default certificate, so the private key is widely deployed.
- Because of the above reasons, it does not protect against man-in-the-middle attacks.
- The certificate has a very long validity period.

The intent of the default certificate is only to provide a means for validating that https is working and/or using in a lab environment if true security is not a concern.

To enable HTTPS with the default server certificate, set the following property in `<install location>\etc\local-properties.xml` of each server hosting a Gateway Node, Data Node, or Apps feature:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
```

OR to set it as a global property via the Search Grid management-console using the command:

```
set-gprop nexidia.searchgrid.https.enabled true
```

Do not set any other `nexidia.searchgrid.https.*` properties when using the default certificate.

Custom Server Certificates

In a production environment, custom, trusted certificates, specific to each server, should be used. Ideally, the certificate chain of trust should go back to a trusted root authority, but the level of trust is customer-determined.

To use any certificate other than the default internal, self-signed certificate, the certificate must be in a keystore. Then that keystore can either be used directly by Search Grid, or imported into Windows internal, secure Certificate

Repository and accessed from there by Search Grid. For the greatest flexibility, a PKCS12 keystore type is recommended.



When generating a certificate or Certificate Signing Request, it should include three SANs (Subject Alternative Names) of type DNS. One should be the FQDN in all-uppercase, one should be the FQDN in all-lowercase, and one should be the IP address of the server (or of the cluster if using clusters). If customer policy prohibits tying a certificate to an IP address and therefore cannot include that SAN, then clients accessing Search Grid will need to have hostname resolution disabled.

To help create and manage custom server certificates, Search Grid includes the Java keytool application from the Java framework. This document describes basic usage of keytool for common situations. Complete keytool documentation is available from Oracle here for Windows:

<http://docs.oracle.com/javase/8/docs/technotes/tools/windows/keytool.html>

Using keytool is not a requirement. Customer-specific processes may use other tools to request/generate certificates and to create keystores, but keytool is provided as an option. To use keytool, open a console and change directory to `<install location>\jre\bin`. Detailed use of keytool is beyond the scope of this document; however, most keytool invocations involve several common concepts:

- The *keystore* is a file used to store the server certificate. Because the keystore contains a private key, its file access permissions should be kept as restrictive as possible. Ideally only the server's administrator and the Search Grid service user should have access to the keystore file. The recommended path for the keystore file is `<data location>\keystore.p12` (or `keystore.jceks` if using that type).
- The *keystore type* defines the format of the keystore file. Although keytool supports many keystore types, Search Grid supports only the JCEKS and PKCS12 types, and PKCS12 is recommended because it provides the most flexibility.
- The *keystore password* is used to access certificates in the keystore. Keytool supports definition of distinct passwords for the keystore and for each certificate in the keystore; however, Search Grid requires that the password for all certificates in keystore be the same as the password for the keystore itself.
- The *certificate alias* is a name used to identify a certificate in the keystore. Search Grid supports only **lowercase** certificate aliases.

Purely as a simple example of how to use keytool, below is an example of creating a new self-signed certificate. However, your own self-signed certificate is no more secure than the default, as described above, and therefore should **not** be used in production, but might be suitable for a lab environment, depending on your lab's security requirements.

To create a new self-signed certificate, run keytool with the following arguments on each server hosting a Search Grid component:

```
keytool -genkey -keystore <data location>\keystore.p12 -storetype PKCS12 -alias <certificate alias> -keyalg RSA
```

In response to the prompt "What is your first and last name?", enter the server's hostname. Responses to subsequent questions will not affect Search Grid functionality. When prompted for the key password, press return to reuse the keystore password.



Remember to restrict read access to the keystore file after creating it.



Keytool also supports generation of CSRs (Certificate Signing Requests) to be sent to a Certificate Authority, and importing such signed certificates into the keystore.



For distributed Search Grid deployments, the simplest approach is to create a single keystore file containing the certificates for all Search Grid servers. (Keytool is one means to do this, by invoking it multiple times, setting a different certificate alias and server hostname each time.) This requires that the alias for each certificate in the keystore be unique. (Using the server name or IP address as the alias are ways to guarantee uniqueness.) The keystore file containing all the certificates would then be copied to each Search Grid server.

The keystore can be used directly by Search Grid, or it can be imported into Windows' internal, secure Certificate Repository, the latter being more secure, but a little more complex to configure.

External Keystore Security

To have Search Grid use a keystore directly, the password to the keystore must be exposed in cleartext within the local-properties.xml file, though access to that file can and should be restricted to only the server's administrator and to the Search Grid service user. Keep in mind, however, that the private key(s) are stored within that keystore, so this may not offer sufficient security to meet specific customers' security policy requirements.

ONLY IF THAT METHOD SATISFIES THE CUSTOMER-SPECIFIC SECURITY REQUIREMENTS, then set the following properties in the `<install location>\etc\local-properties.xml` file on each server hosting a Search Grid component:

```
<entry key="nexidia.searchgrid.https.keystore.type">[keystore type]</entry>
<entry key="nexidia.searchgrid.https.keystore.location">[data location]\[keystore file name]</entry>
<entry key="nexidia.searchgrid.https.keystore.password">[keystore password]</entry>
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">[certificate alias]</entry>
<entry key="nexidia.searchgrid.https.enabled">true</entry>
```

The [keystore type] must be either JCEKS or PKCS12, for example:

```
<entry key="nexidia.searchgrid.https.keystore.type">PKCS12</entry>
```

Specify the keystore type that matches the keystore you created. (Note: If using the default certificate provided by Search Grid, do not set these properties and refer to the [Default Certificate](#) section.)



If the keystore contains multiple certificates, be sure to use the correct certificate alias for each server.



The keystore password is stored in cleartext, so the local-properties.xml file's and the keystore file's read permissions should be restricted to the administrator and the Search Grid service user.

Windows Keystore Security

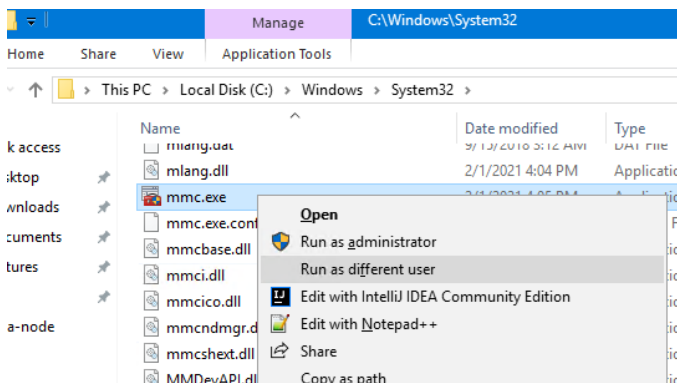
WHEN ADDITIONAL SECURITY IS REQUIRED beyond that provided by the External Keystore Security method, the keystore will need to be imported into Windows' internal secure Certificate Repository, eliminating the cleartext keystore password that the External Keystore Security method exposes. This requires that keystore be in PKCS12 (.p12) format.

Note: If you already have a JCEKS keystore and wish to convert it to a PKCS12 keystore, you can use keytool to do that:

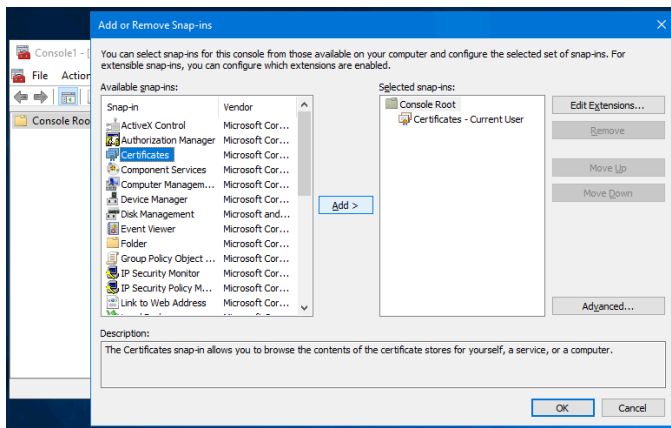
```
keytool -importkeystore -srckeystore <YourCurrentKeystoreFile.jceks> -srcstoretype JCEKS -deststoretype PKCS12 -destkeystore <YourNewKeystoreFile.p12>
```

To import a keystore into Windows, *it must be imported as the Search Grid service user*. Use the following procedure:

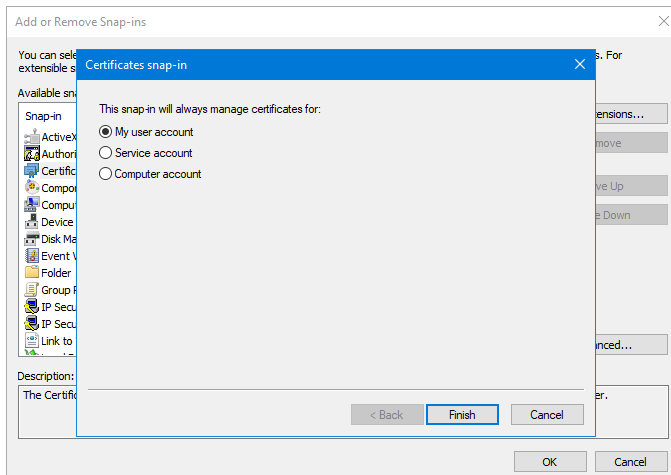
- In the Windows search bar, type **MMC**, then right-click on the **MMC** icon and choose "Open file location". (Alternatively, in Explorer, navigate to C:\Windows\System32 and scroll down to mmc.exe)
- **SHIFT+RightClick** on the mmc.exe icon shown in the Explorer window, and choose "Run as different user".



- Enter the service user's name and password when prompted.
- Click **File** → **Add/Remove Snap In**

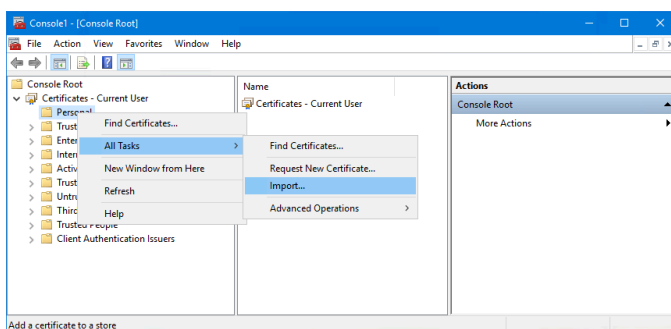


- In the Available snap-ins list at the left, select **Certificates** then click on **Add**.
- If prompted for the account type...

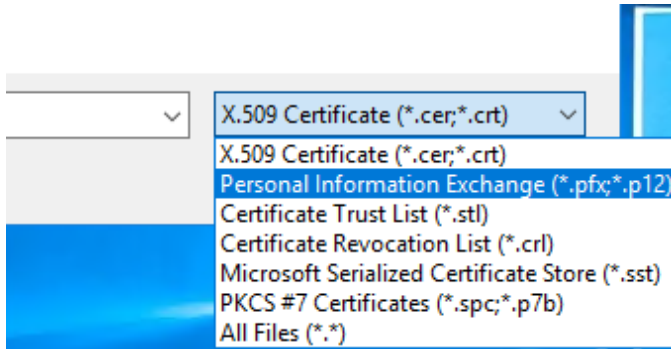


choose **My User Account** then click **Finish** (prompting depends on permissions of the service user, and you likely will not be prompted)

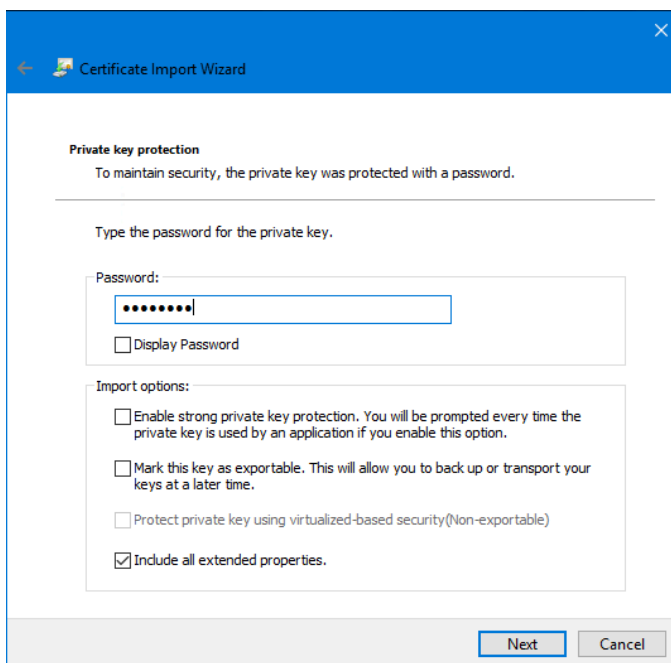
- Click **OK** to close the Add or Remove Snap-ins dialog.
- Expand the "Certificates – Current User" item by clicking the arrow to the left of it.
- **RightClick** on the **Personal** folder under "Certificates – Current User", then choose **All Tasks** → **Import**



- An Import Wizard dialog should appear with Current User selected. Click **Next**.
- Click **Browse**.
- In the Open dialog, near the bottom-right, next to the File name field, **click on the drop-down list**.



- Choose "**Personal Information Exchange (*.pfx;*.p12)**"
- Navigate to the location of your keystore file and select your keystore's .p12 file, then click **Open**, then click **Next**.

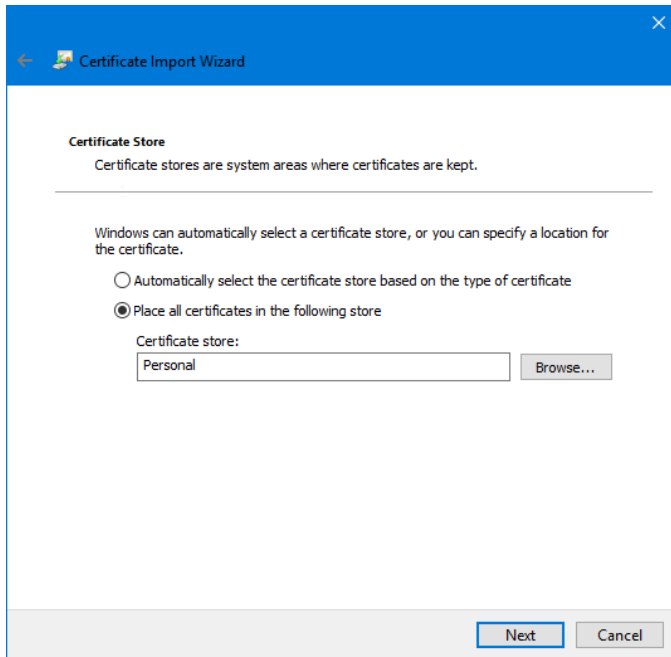


- **Type** in the password to your keystore.
- In the "*Import options*" section, ensure **ONLY** the "**Include all extended properties**" box is checked.



If the "Enable strong private key protection." box is checked, that will cause Windows to prompt for the password every time the key is accessed by Search Grid, blocking operation. **DO NOT CHECK THAT BOX.**

- Click **Next**.



- If not already chosen by default, choose the **"Place all certificates in following store"** option, and set the **"Certificate store"** field to **Personal**, then click **Next**.
- Verify the information displayed looks correct, then click **Finish**.

If you go into the Certificates folder under Personal, then double-click on the certificate, it should show the friendly name (what you set for the "alias" when adding the certificate to the keystore). If the certificate was signed by a trusted Certificate Authority, or if it was signed by the organization and the organization has its own certificated (used to sign this certificate) in the server's Trusted Root Certification Authorities location, then the chain of Trust should not be broken. You can verify this in the Certification Path. Also verify the Subject and Subject Alternative Names in the Details tab is also correct.

- Configure Search Grid to use the certificate that was imported into Windows by setting the following properties in the `<install location>\etc\local-properties.xml` file on each server hosting a Search Grid component:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
<entry key="nexidia.searchgrid.https.keystore.type">Windows-MY</entry>
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">[your certificate's alias]</entry>
```

For example, if your certificate alias (also the "friendly name" as shown in the Windows UI) is 10.1.55.123, the last item above would be:

```
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">10.1.55.123 </entry>
```



It is important to note that certificates expire and can also be revoked. This means that periodically (typically every year or more), the certificate will need to be updated on all Search Grid servers. If Search Grid is using the keystore directly, then just that file needs to be updated. If Search Grid is using the certificate stored within Windows, then the new certificate will have to be added to Windows for the Search Grid service user, just as the original one was, above.

Search Grid Client Considerations

Search Grid's own client applications, *search-console* and *management-console* can use HTTPS, and if HTTPS is enabled on the gateway/control node(s), then all nodes from which those clients can be run need to have HTTPS enabled.

The Disaster Recovery feature is also a client, but a client to the Backup instance's gateway/control node. If *and only if* the backup instance has HTTPS enabled, then ALL Data Nodes of the primary instance **must** have HTTPS enabled.

- If HTTPS is being used by clients only to secure data over the network through encryption, and the clients do not need to authenticate the servers they're connecting to, then only the following property needs to be set in the `<install location>\etc\local-properties.xml` file of every node which contains one of the aforementioned clients:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
```

- If the clients need to authenticate the servers they are connecting to, then refer to the [Client Authentication of Servers](#) section.

Client Authentication of Servers

For client applications to authenticate the server, they need to trust the server's certificate or another certificate in the server's chain of trust. Unless you are using custom certificates issued by a Certificate Authority that has a chain of trust all the way to a trusted root authority, this involves exporting the server's public key in a certificate that client applications can consume. To export the certificate to a file (usually with a ".cer" extension), invoke keytool similar to as follows, but specifying your specific keystore file name and storetype (PKCS12 or JCEKS):

```
keytool -exportcert -file <certificate file> -keystore <data location>\keystore.p12 -storetype PKCS12 -alias <certificate alias>
```

Note that the exported certificates (.cer files) *do not* contain the private keys required for decryption. Those keys remain private, and typically they are contained within the keystore into to which you stored the keypair when the keypair was generated and imported the certificate into after it had been signed. The exported certificates are used by the clients only to authenticate the servers that use them.

Export the certificates for all deployed Search Grid servers and supply the resulting files to client application developers. The mechanism for using the exported certificates in the client application depends on the development framework being used.

Configuring Search Grid's Clients to Authenticate Servers

For Search Grid's own client applications, *search-console*, *management-console*, as well as the Disaster Recovery feature, several properties must be set and public certificates from all the servers to which they connect must be made available to the clients.



The Disaster Recovery feature on the *Primary* instance is a client to gateway node(s) on the *Backup* instance. Therefore, if using Disaster Recovery, *and* using HTTPS on the Backup instance, *and* enabling clients to authenticate servers, then **ALL** the Data Nodes on the *Primary* instance will require the certificate(s) of the *Backup* instance's gateway node(s).

Direct Keystore Use

If you configured Search Grid to use a keystore directly for the server's own certificate (lower security due to keystore password exposed in cleartext), you will need to import all of those *other* servers' certificates exported above into that single keystore. For example, using `keytool`:

```
keytool -importcert -file <certificate file> -keystore <data location>\<keystore name> -storetype <keystore type> -alias <certificate alias>
```

The choice of certificate alias is not important so long as it is lowercase and unique within the keystore.



If a machine that runs Search Grid clients is also acting as a Search Grid server (for example, the Gateway, on which the *management-console* app may be run), then import those *other servers'* certificates into the same keystore that contains that server's certificate and keypair. The alias for these imported certificates to authenticate against is not important; only the alias for the server's own certificate is. The end result should be a keystore containing that server's certificate and corresponding keypair *plus* the certificates of all the other servers.



If there is some need to have Search Grid apps such as *management-console* and *search-console* not authenticate servers while Data Nodes for Disaster Recovery do, or visa-versa, then **instead of configuring everything in a machine's `<install location>\etc\local-properties.xml` file as shown below**, you would have to set the `clientsAuthenticateServers` value you want for apps in the `<install location>\etc\apps\apps-properties.xml` file on each machine, and the `clientsAuthenticateServers` value you want for Data Nodes to use for Disaster Recovery in the `<install location>\etc\data-node\data-node-properties.xml` file on each machine. (Simplest is to configure them the same via the `<install location>\etc\local-properties.xml` as below.)

- Configure Search Grid to access that keystore by setting the following properties in the `<install location>\etc\local-properties.xml` file on each machine on which a Search Grid client may be run:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
<entry key="nexidia.searchgrid.https.clientsAuthenticateServers">true</entry>
<entry key="nexidia.searchgrid.https.keystore.type">[keystore type]</entry>
<entry key="nexidia.searchgrid.https.keystore.location">[data location][keystore file name]</entry>
<entry key="nexidia.searchgrid.https.keystore.password">[keystore password]</entry>
```

- Only if that machine is also acting as a server, include this setting (explained in more detail in [Custom Server Certificates](#) section):

```
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">[your certificate's alias]</entry>
```

Windows Certificate Repository Use

Using the Windows Certificate Repository is beneficial in that it does not require exposing a keystore password in cleartext; however, it can get complicated when enabling Client Authentication of Servers, particularly on a machine that is running one or more Search Grid services and on which Search Grid apps, such as *management-console* may be run. This is due to a combination of how Windows' certificate repository works and how Java access that repository.

- For the Search Grid Services, that machine's certificate is stored specifically for the service user's account. This is what is described in the [Windows Keystore Security](#) section. If the Disaster Recovery feature is being used, then all Data Nodes on the *Primary* instance must also have the certificate (.cer file) of the *Backup* instances Gateway Node(s) installed into the Windows Certificate Repository for the service user's account because the base-service on the Data Nodes will need to authenticate the Backup instance's Gateway Node(s). To do this, follow the instructions in the [Windows Keystore Security](#) section **except** where it says to import the certificate from a .p12 keystore, instead choose to import the certificate(s) directly from the .cer file(s) exported from the Backup instance's Gateway Node(s).
- For apps, such as *management-console* that are being run as users other than the service user, the certificates stored for the service user's access will not be visible. The certificates of all the Search Grid nodes *that the apps need* to authenticate (typically only the Gateway/Control Node(s)) must be either:
 - imported into the "Personal" folder of the certificate repository for **each user** of the apps (Option A below)

OR

- imported to the Trusted Root Certification Authority of the **machine** (not any specific users), but then the apps will have to be configured separately from the node's services rather than using a common configuration in the machine's `<install location>\etc\local-properties.xml` file (Option B below).



To import certificates for other users, launch MMC as described in the [Windows Keystore Security](#) section, but specify the user(s) you need instead of the service user.

To import certificates for the machine, launch MMC using "Run as administrator", and when adding the snap-in, choose "Computer account".

- **Option A:** For each Search Grid node, if no Search Grid apps will be run by users other than the service user on a machine **OR** you've imported all the relevant certificates (i.e. this machine's certificate the Gateway/Control node(s) certificate(s), and if using the Disaster Recovery feature, the Backup instance's Gateway node(s) certificate(s)) into Windows' Certificate Repository, then you set the following values in the machine's

<install location>\etc\local-properties.xml file:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
<entry key="nexidia.searchgrid.https.clientsAuthenticateServers">true</entry>
<entry key="nexidia.searchgrid.https.keystore.type">Windows-MY</entry>
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">[your certificate's alias]</entry>
```

- For more about the certificate alias, see the [Custom Server Certificates](#) section
- **Option B** For each Search Grid Node that has its machine's certificates, and if using the Disaster Recovery feature, the Backup instance's Gateway node(s) certificate(s), imported into the **service user's** "Personal" folder of the certificate repository for the services to access, but has all the **other** relevant certificates (typically only the Gateway/Control Node(s)) AND this machine's certificate imported into the **machine's** "Trusted Root Certification Authorities":

- Set the following values in the machine's <install location>\etc\apps\apps-properties.xml file

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
<entry key="nexidia.searchgrid.https.clientsAuthenticateServers">true</entry>
<entry key="nexidia.searchgrid.https.keystore.type">Windows-ROOT</entry>
```

- Set the following values in each of the machine's <install location>\etc\<NodeType>\<NodeType>-properties.xml files, for example, on a Data Node, the <install location>\etc\data-node\data-node-properties.xml file, or on a Gateway/Control Node, both the <install location>\etc\gateway-node\gateway-service-properties.xml file **and** the <install location>\etc\control-node\control-service-properties.xml file:

```
<entry key="nexidia.searchgrid.https.enabled">true</entry>
<entry key="nexidia.searchgrid.https.clientsAuthenticateServers">true</entry>
<entry key="nexidia.searchgrid.https.keystore.type">Windows-MY</entry>
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">[your certificate's alias]</entry>
```

- For more about the certificate alias, see the [Custom Server Certificates](#) section

Management Console Commands

The Management Console application supports changing and monitoring the system mode. These commands are available only to the admin user.

- To display the current system mode:

```
# get-mode
```

- To change the system mode:

Adding or Decommissioning Data Nodes & Data Redistribution



Unlike older versions of Search Grid which required services to be stopped in order to run data redistribution, it can now be run live alongside normal operations. It is recommended to run at times with lower system workload in order to have the least impact on day-to-day operations.

As the data volume in a Search Grid installation increases, it may become necessary to install additional Data Nodes. After a new Data Node is installed, it will gradually receive data as new media are declared; however, because the new node starts with no data, the new node will not yield any immediate benefit to search performance. To maximize use of the new Data Node, the administrator must perform data redistribution as described below. After data redistribution is complete, the new Data Node will contain an equal share of phonetic data.

Data redistribution is also used to decommission a Data Node from an existing Search Grid instance. When decommissioning a node, data is redistributed away from the Data Node to be removed and into Data Nodes that will remain in the grid.

1. *If adding a new Data Node:* Install the new Data Node and customize its configuration settings as needed. Please see the [Installation Procedure](#) section for details, including service restart requirements.
2. *If decommissioning a Data Node:* Set `nexidia.searchgrid.node.capacity` to `0.0` in `data-node-properties.xml` on the node to be decommissioned. Note: That property may or may not already exist in the `data-node-properties.xml` file. If it exists, modify it, otherwise add the property. (See [Data Node](#) section for details.) After setting the capacity to `0.0`, the base and agent services on that Data Node must be restarted, and the gateway service on the Gateway Node(s) must be restarted.
3. Use Management Console to start redistribution by entering "req-redist".
 - If redistribution is needed, the console displays a list of which nodes are sending media to which other nodes.
 - Search Grid begins moving phonetic indexes and other data between Data Nodes until data volume is balanced across all nodes.
 - Redistributed media may need to be remonogested, and if so, that happens automatically in the background and continues, if needed, after redistribution is complete.
 - The Management Console "get-redistin" command may be used to check redistribution progress.
 - To stop redistribution, use the Management Console "cancel-redist" command. Any redistribution progress made so far will be retained and Search Grid will continue its normal operations.

If not decommissioning any nodes, there is nothing more required. However, if you are decommissioning any nodes, follow steps 4..9

4. Periodically use the Management Console "get-redistin" command to check redistribution progress.

5. When redistribution is complete, stop all Search Grid services on all servers *except* the Control Service.
6. In Management Console, use the "set-cred" command and enter primitive mode.
7. Use Management Console to modify global properties.
 - Remove the Data Node's name from `nexidia.searchgrid.nodeNameList`. (See the [Global Properties](#) section for details.)
 - Stop all Search Grid Services on the decommissioned node.
 - Uninstall Search Grid from the decommissioned node, or at minimum, set all services to Disabled via Windows' Services UI.



The above steps are critical for decommissioning a node. Failure to do them could prevent Search Grid from operating, believing it is in a constant "initializing" state.

8. Start all Search Grid Services on all servers.
9. Resume normal use of Search Grid. Note that it may take an hour or more to reclaim storage space consumed by migrated data.

Depending on the data volume that needs redistribution, and the number of nodes involved, redistribution can take hours or even days. During this time, Search Grid may see impacts on performance. The option to cancel redistribution (step 8) can be used to alleviate any performance issues.

Note that redistribution is beneficial *only* when Data Nodes are added or removed. It is not necessary to run redistribution on a regular basis.

Adding or Removing Gateway and Control Nodes

Typically, Gateway Nodes reside on the same server(s) as Control Nodes, but that is not a requirement. As mentioned in the [Large Distributed Deployment](#) section, Search Grid can have 1, 2 or 3 Gateways; however it can have *exactly* 1 or *exactly* 3 Control Nodes. In practice, the number of Gateway Nodes generally matches the number of Control Nodes, usually on the same servers as the Control Nodes.

Gateway Nodes

Gateway Nodes contain no data and therefore can be added and removed at will, so long as there is always at least 1 Gateway for the client (i.e. NIA) to connect to.

Control Nodes

Control nodes store data that is used by *every type of node* in Search Grid (thus the word "control" in their name), and as such, great care must be taken to prevent data loss when adding or removing Control Nodes. Please follow the

instructions below very carefully, and if there are any questions, please contact NiCE R&D *before* beginning the process.

PREREQUISITES & RESTRICTIONS for Adding or Removing Control Nodes:

1. The client (i.e. NIA) **must** be stopped or paused. It is essential that no media is ingested / declared at any point from before the beginning until after the end of the processes below.
2. No other changes to Search Grid (adding language or phrase packs, changing passwords, etc. — nothing) are allowed from the start of the process until the end.
-  3. The process must be completed once started. DO NOT bring Search Grid up mid-process, thinking it can be stopped and the process resumed later. Be certain to allow enough time to complete the process. Depending on the total number of nodes of all types that are in the grid, this could be a lengthy process because it involves editing the local-properties.xml file on every server containing Search Grid services.

Failure to heed this warning will likely result in Data Nodes suffering from serious data loss or corruption.

Removing Control Nodes

Switching from 3 control nodes to 1 is relatively simple, but care must still be taken in order to prevent data loss or corruption. Also, remember that configuring Search Grid to have 2 control nodes is not a supported configuration. The point of there being multiple control nodes is so that Search Grid can function so long as a majority of Control Nodes are up. With 2 nodes, that benefit is lost because 1 of 2 is not a majority.

Determine if Removal is Safe

1. Ensure the client (i.e. NIA) is stopped or paused, guaranteeing no media declarations will be attempted. (Do not stop any Search Grid services yet.)
2. Verify all the existing control nodes are in sync:
 - a. On any Search Grid node, open <install location>\etc\local-properties.xml in any text editor, and find the line containing the connectionString. It will look *similar* to this:

```
<entry key="nexidia.searchgrid.control.connectionString">10.1.47.140,10.1.51.227,10.1.51.215</entry>
```
 - b. Copy the IP addresses. (From the above example, that is the 10.1.47.140, 10.1.51.227, 10.1.51.215 portion.)
 - c. Open a PowerShell window.

- d. At the PowerShell prompt "cd" to your <install location>\tools folder.
- e. At the PowerShell prompt, run the following command:

```
| > .\Test-Zookeeper.ps1 -ControlString "<the IP addresses you copied above>"
```

Note that quotations are required around the IP addresses. For example, similar to this:

```
| .\Test-Zookeeper.ps1 -ControlString "10.1.47.140,10.1.51.227,10.1.51.215"
```

You should see something similar to this:

```
| CHECKING 10.1.51.227...
|     zk_server_state leader
| zk_peer_state  leading - broadcast
|     zk_znode_count 47
|     zk_synced_followers 2
| zk_synced_non_voting_followers 0
|     zk_learners 2
|     zk_quorum_size 3
| zk_unrecoverable_error_count 0
|     zk_restore_error_count 0
|     zk_snapshot_error_count 0 +
| CHECKING 10.1.50.71...
|     zk_server_state follower
| zk_peer_state  following - broadcast
|     zk_znode_count 47
|     zk_quorum_size 3
| zk_unrecoverable_error_count 0
|     zk_restore_error_count 0
|     zk_snapshot_error_count 0
| CHECKING 10.1.50.97...
|     zk_server_state follower
| zk_peer_state  following - broadcast
|     zk_znode_count 47
|     zk_quorum_size 3
| zk_unrecoverable_error_count 0
|     zk_restore_error_count 0
|     zk_snapshot_error_count 0
```

Verify that for the node that has a zk_server_state of "leader", the number of zk_synced_followers is 2.

If the number of zk_synced_followers is not 2, do not continue without first consulting NiCE R&D



If you need to start the client back up in the meantime, that is fine. No changes have been made to Search Grid. If you do start back up, you will need to repeat the "Determine if Removal is Safe" steps again after consulting with R&D.

Perform Removal

Do NOT proceed with removal without first completing the "Determine if Removal is Safe" steps.

1. Ensure the client (i.e. NIA) is stopped or paused, guaranteeing no media declarations will be attempted. (Do not stop any Search Grid services yet.)
2. Stop ALL Search Grid services of ALL types on ALL nodes.
3. On **each of the two (2) servers** containing the two Control Nodes you wish to decommission, you must either uninstall the Control Service, or disable it and then may uninstall it at a later time.
 - To disable the control service...
 - i. Using the Windows "Services" UI, right-click on the NiCE Search Grid Control Service, and choose [Properties](#).
 - ii. Change the "Startup type" option to [Disabled](#).
 - iii. Click [OK](#).
 - To uninstall the Control service...
 - i. Using the Windows "Add or Remove Programs" UI, find NiCE Search Grid, click on it, and then click on the [Modify](#) button to launch the Search Grid Setup Wizard.
 - ii. Click on [Next](#) on the Setup Wizard's Welcome screen.
 - iii. Click on the [Change](#) button.
 - iv. Click on the icon next to where it says Control Node and choose the "Entire feature will be unavailable" option.

v. Click on [Next](#) and then follow any prompts until it completes.

4. On EVERY server that is running ANY Search Grid services, use any text editor (using "Run as Administrator" when launching the editor) to open `<install location>\etc\local-properties.xml`, and find the line containing the `connectionString`. It will look *similar* to this:

```
<entry key="nexidia.searchgrid.control.connectionString">10.1.47.140,10.1.51.227,10.1.51.215</entry>
```

5. Remove the two IP addresses of the two Control Nodes that you disabled or uninstalled. (That line should now contain just one IP address.)
6. Save the file.

Repeat until ALL servers have been modified.

7. Start ONLY the one remaining Control service.
8. Verify it is functioning...

- a. Open a PowerShell window.
- b. At the PowerShell prompt "cd" to your `<install location>\tools` folder.
- c. At the PowerShell prompt, run the following command:

```
> .\Test-Zookeeper.ps1 -ConnectionString "<the IP address of your Control Node>"
```

- d. Verify that the `zk_server_state` is "standalone".
- e. Run the Search Grid management-console app, and start it in native mode by pressing [Enter](#) when prompted for a user (instead of entering a user).
- f. Enter `get-gprops` then press [Enter](#).
- g. Verify that the value reported for `nexidia.searchgrid.nodeNameList` contains the names of all the data nodes, and if there are any additional global properties set, that they all look correct.



If anything in these "Verify it is functioning" steps does not look correct DO NOT start up any other services, and contact NiCE R&D immediately.

9. If everything looks fine, you may now start up ALL Search Grid services on ALL nodes (except the two control services that you either uninstalled or disabled).



If you only disabled the Control services on the Control Nodes you've decommissioned, it is highly recommended that you uninstall them now, **AND delete the <data location>\control-node folder on those decommissioned Control Nodes** to prevent ever running that service and using old control data in Search Grid, as that would cause duplicate media IDs and cause other corruption. At the very least, please delete that data.

Adding Control Nodes

The process of expanding from one to three control nodes is fairly simple, but must be adhered to. Do not take the steps lightly. Be slow and methodical.



When expanding from 1 Control Node to 3 Control Nodes, great care must be given to follow the exact procedure below, otherwise some or all of the data stored on the original control node may be lost and unrecoverable.

1. Ensure the client (i.e. NIA) stopped or paused, guaranteeing no media declarations will be attempted.
2. Stop ALL Search Grid services on ALL servers.
3. On the current Control Node (referred to as the "original Control Node" hereafter) make a backup copy of the entire <data location>\control-service\control-data directory and all its subdirectories.
4. Install two new Control Services on the new servers.
5. On the two new servers, **delete** the entire <data location>\control-service\control-data directory and all its subdirectories. This is critical, even if you just performed a fresh installation. The entire <data location>\control-service\control-data directory must be completely gone.



DO NOT START THE CONTROL SERVICE ON ANY SERVER. At this point, ALL Search Grid services should be stopped, the original Control Service's control-data should have been backed up (but remain), and the control-data from the two new servers should be completely removed.

6. Get the IP addresses of the two new servers.
7. On the original Control Node, **edit** the <install location>\etc\local-properties.xml file with any text editor. You will need to run that editor "as administrator", however, in order to save changes.
8. Locate the `controlConnectionString` property in the file. It will look similar to this, where the IP address shown will be the IP address of your *original* Control Node:

```
| <entry key="nexidia.searchgrid.control.connectionString">10.1.47.140</entry>
```

9. **Modify** that line, appending the two IP addresses of the two new Control Nodes, separated by commas (no spaces). For example, if the IP address of your original Control Node is 10.1.47.140, and the IP addresses of your

two new Control Nodes are 10.1.51.212 and 10.1.51.214, then the modified line would look like this:

```
<entry key="nexidia.searchgrid.control.connectionString">10.1.47.140,10.1.51.212,10.1.51.214</entry>
```

10. **Copy** that line. You will need it many times again.
11. **Save** the file.
12. On *both* of the two new Control Nodes, edit their respective `<install location>\etc\local-properties.xml` files, and **replace** their connectionString lines with the *identical* line copied from the original Control Node's file, saved in the step above.
13. **Save** those files on both of the two new Control Nodes.
14. Open a PowerShell window *as administrator* on the original Control Node.
15. Change directories (cd) to the `<install location>\tools` directory.
16. Start the ORIGINAL Control Node's Control Service ONLY. **Do not start any other Search Grid service on that or any other server. This is critical.**
17. At the PowerShell prompt, execute the following command, but substitute the IP addresses of your three Control Nodes. Note that there must be single or double quotes around the IP list. Using the example IPs from above, the command would be this:

```
.\Test-Zookeeper.ps1 -ControlString '10.1.47.140,10.1.51.212,10.1.51.214'
```

18. You should get a response stating that "This Zookeeper instance is not currently serving requests". It will look similar to this:

```
CHECKING 10.1.47.140...
This ZooKeeper instance is not currently serving requests
CHECKING 10.1.51.212...
ERROR: Exception calling ".ctor" with "2" argument(s): "No connection could be made because the target
machine actively refused it 10.1.51.212:25134" +
CHECKING 10.1.51.214...
ERROR: Exception calling ".ctor" with "2" argument(s): "No connection could be made because the target
machine actively refused it 10.1.51.214:25134"
```

If it does not report "This Zookeeper instance is not currently serving requests" for the original Control Service's IP address and report connection failures to the two new IP address, STOP. DO NOT CONTINUE. Under no circumstances start the new Control Services.



If you need to get Search Grid back on line:

- Stop the Control Service.

- Revert the connectionString in the original Control Service's `<install location>\etc\local-properties.xml` file so that it contains only the original Control Service's IP address and save the file.
- Using Windows Services UI on the two new Control Nodes, go into the Control Service's properties and set them to `Disabled` so they cannot be accidentally started. (As an additional precaution, you may want to disconnect the new servers from your network.)
- Only once you've done the above steps, you may restart all Search Grid Services on all servers EXCEPT THE NEW ONES.

Only if everything went fine with step 18, you may continue.

19. On ONE of the new Control Nodes, verify that the `<data location>\control-service\control-data` directory does not exist (you should have deleted it at step 5), and only once you have double-checked that it does not exist, start the Control Service ONLY. **Do not start any other Search Grid service on that or any other server. This is critical.**
20. At the PowerShell prompt that is open on the original Control node, run the same command you performed at step 17.
21. You should get a response similar to this, where the ORIGINAL Control Node is the "leader", and the new control node you started is a "follower", and the other new one that you have not started still reports a connection error:

```
CHECKING 10.1.47.140...
zk_server_state leader
zk_peer_state leading - broadcast
zk_znode_count 53
zk_synced_followers 1
zk_synced_non_voting_followers 0
zk_learners 1
zk_quorum_size 3
zk_unrecoverable_error_count 0
zk_restore_error_count 0
zk_snapshot_error_count 0
CHECKING 10.1.51.212...
zk_server_state follower
zk_peer_state following - broadcast
zk_znode_count 53
zk_quorum_size 3
zk_unrecoverable_error_count 0
zk_restore_error_count 0
zk_snapshot_error_count 0
CHECKING 10.1.51.214...
ERROR: Exception calling ".ctor" with "2" argument(s): "No connection could be made because the target
machine actively refused it 10.1.51.214:25134"
```

If your original Control Node is not the leader, STOP. DO NOT CONTINUE. Call for R&D support immediately.

If you need to get Search Grid back online:



- Stop the Control Service on **both** Control Nodes.
- Disable both of the new Control Nodes as mentioned in the earlier warning at step 18.
- Revert the original Control Nodes connectionString as mentioned in the earlier warning at step 18
- Restore the control-data to the original Control Node that you backed up at step 3.
- Start ONLY the Control service of your original Control node.
- Run the Test-Zookeeper.ps1 script again, exactly has you had done above. It should report a status of "standalone" for the original control node and connection failures for the others.
- Only once you've done the above steps, you may restart all Search Grid Services on all servers EXCEPT THE NEW ONES.

Only if everything went fine with step 21, you may continue.

22. On the other of the new Control Nodes, verify that the `<data location>\control-service\control-data` directory does not exist (you should have deleted it at step 5), and only once you have double-checked that it does not exist, start the Control Service ONLY. **Do not start any other Search Grid service on that or any other server. This is critical.**
23. At the PowerShell prompt that is open on the original Control node, run the same command you performed at step 17.
24. You should get a response similar to this, where the ORIGINAL Control Node is the "leader", and the BOTH of the new control nodes are "followers":

```
CHECKING 10.1.47.140...
zk_server_state leader
zk_peer_state leading - broadcast
zk_znode_count 56
zk_synced_followers 2
zk_synced_non_voting_followers 0
zk_learners 2
zk_quorum_size 3
zk_unrecoverable_error_count 0
zk_restore_error_count 0
zk_snapshot_error_count 0
CHECKING 10.1.51.212...
```

```
zk_server_state follower
zk_peer_state following - broadcast
zk_znode_count 56
zk_quorum_size 3
zk_unrecoverable_error_count 0
zk_restore_error_count 0
zk_snapshot_error_count 0
CHECKING 10.1.51.214...
zk_server_state follower
zk_peer_state following - broadcast
zk_znode_count 56
zk_quorum_size 3
zk_unrecoverable_error_count 0
zk_restore_error_count 0
zk_snapshot_error_count 0
```

If your original Control Node is not the leader, STOP. DO NOT CONTINUE. Call for R&D support immediately.

If you need to get Search Grid back online:



- Stop the Control Service on **all three** Control Nodes.
- Disable both of the new Control Nodes as mentioned in the earlier warning at step 18.
- Revert the original Control Nodes connectionString as mentioned in the earlier warning at step 18
- Restore the control-data to the original Control Node that you backed up at step 3.
- Start ONLY the Control service of your original Control node.
- Run the Test-Zookeeper.ps1 script again, exactly as you had done above. It should report a status of "standalone" for the original control node and connection failures for the others.
- Only once you've done the above steps, you may restart all Search Grid Services on all servers EXCEPT THE NEW ONES.

Only if everything went fine with step 24, you may continue.

25. Start the Search Grid management-console app on any of the three Control Node servers.
26. When prompted for a user, press Enter (do not enter a user) to go into native mode.
27. Run the "get-gprops" command and verify that the nodeNameList and any other global properties you had set for your grid are correct.

If your global properties are not correct, STOP. DO NOT CONTINUE. Call for R&D support immediately.

If you need to get Search Grid back online:

- Stop the Control Service on **all three** Control Nodes.
- Disable both of the new Control Nodes as mentioned in the earlier warning at step 18.
- Revert the original Control Nodes connectionString as mentioned in the earlier warning at step 18
- Restore the control-data to the original Control Node that you backed up at step 3.
- Start **ONLY** the Control service of your original Control node.
- Run the Test-Zookeeper.ps1 script again, exactly has you had done above. It should report a status of "standalone" for the original control node and connection failures for the others.
- Only once you've done the above steps, you may restart all Search Grid Services on all servers **EXCEPT THE NEW ONES**.



Only if everything went fine with step 27, you may continue.

28. Exit the management-console app. (Type "exit", then press Enter.) For ALL servers containing Search Grid services — gateway nodes, data nodes, compute nodes, whatever your grid's configuration is — you must **edit** the `<install location>\etc\local-properties.xml` file on each server with any text editor (run that editor "as administrator" in order to save changes) and **replace** the connectionString property with **EXACTLY** the same property that you set in the three Control Nodes.
29. Double-check to make sure you have properly updated the `<install location>\etc\local-properties.xml` file on ALL Search Grid servers. **This is very important.** If you missed one, the grid might work initially, but could stop working in the future if any services were restarted in a different order, or one of the Control nodes went down temporarily.
30. Once you are certain that the same value is set for `nexidia.searchgrid.control.connectionString` in the `<install location>\etc\local-properties.xml` file on ALL Search Grid servers, you may start up all Search Grid services on all servers.
31. Once all services have started, it is recommended to perform a test-system command to verify that Search Grid is fully functional.

Server Migration

If it becomes necessary to move a Search Grid node to a new server, for example, if the hardware has become obsolete, this can be done using the following method:

1. Install *the same version* of Search Grid onto the new server, but DO NOT START any Search Grid services.
2. Stop **ALL** Search Grid services on **ALL** nodes. (Do not only stop the node being migrated.)
3. On the **old** server, use the offline backup procedure (see [Backups](#) section) to back all data (files and folders) from the old server's <data location> folder.
4. On the **new** server, remove all files and subdirectories from the new server's <data location> folder if any exist. ***This is critical.***
5. On the **new** server, restore the backed-up data to the new server, into its <data location> folder.
6. On the **new** server, for each node type in the <data location> (for example data-node, control-node, gateway-node, etc.) **delete all *.uuid files**. This is important because the new server must create its own, new, unique *.uuid files when it is first started, *not* use the *.uuid files from the old server.
7. On the **old** server, back up everything that is in the <install location>\etc folder, being sure to preserve permissions by using xcopy or robocopy and their respective options, as indicated in the [Backups](#) section.
8. On the **new** server, restore that etc data into the <install location>\etc folder, being sure to preserve permissions by using xcopy or robocopy and their respective options, as indicated in the [Backups](#) section.
9. If the new server's <data location> is different from that of the old server (for example, was `C:\ProgramData\Nexidia\Search Grid` on the old server, but is `G:\Search Grid Data` on the new server), you must edit each <node-or-service-type>-properties.xml (for example, data-node-properties.xml, base-service-properties.xml, etc.) and anywhere that specifies the old server's <data location>, you must change it to specify the new server's <data location>.
10. Start ALL services on ALL nodes, ***except on the old server.***
11. Verify Search Grid is functioning using the Search Grid management-console app:
 - Use the management-console's "get-systop" command to verify all nodes, including the new server, are shown.
 - Use the management-console's "test-system" command to verify all tests pass.
12. On the **old** server, uninstall Search Grid from the old server, or at minimum, set all Search Grid services to Disabled via Windows' Services UI. (The old node must not be allowed to present itself to Search Grid's other nodes.)



If the old and new servers are both connected to the network, instead of backing up and restoring, you can simply copy from the old server to the new server, *however*, it is still *essential* to preserve permissions by using xcopy or robocopy and their respective options, as mentioned in the [Backups](#) section.

Backups

Administrators can back up Search Grid with standard file-system based backup tools. Backups must be performed offline, which requires all Search Grid services to be stopped before the backup is started.

- Backup and restore operations apply to all Search Grid nodes except Compute Nodes. Consider a system comprising two Data Nodes, one Gateway Node, and several Compute Nodes. If one Data Node experiences a catastrophic failure, then data on both Data Nodes and the Gateway Node must be restored from the latest backup.
- Backup and restore operations apply to data for all tenants.
- Use a file copy utility that preserves file ownership and permission attributes. Many common "zip" tools do not. If you choose to use these zip tools, make sure you have a way of restoring file ownership and permission attributes. (The xcopy command can be run with the /O option to preserve file ownership and permission attributes; the /COPYALL flag performs a similar function for robocopy.)
- Any files in the backup directories that are not in the node data locations must be deleted. Some file copy utilities call this "mirroring."

In the case of a non-catastrophic failure, in which only individual files are lost, a global restore may not be required.

- Files in the "phonetic-data" data subdirectories can be restored on an individual basis.
- Files in the "web-server", "temp-media", and "logs" folders do not need to be restored. In case of a problem with files in these locations, simply delete the folder, and it will be recreated when the service is restarted.

To perform a backup, follow these steps:

1. Stop all Search Grid services on all servers.
2. Back up the entire contents of all node data locations on each server. If you have specified different directories for specific data directories (e.g. pat, polygest; as described in the Common Custom Configuration and Configuration Property Reference sections), make sure those locations are included in the backup.
3. Start all Search Grid services on all servers.

To restore a backup, follow the same steps as backing up, except restore. Similar to backing up, you must restore the entire contents of a backup.



If you are using the Disaster Recovery feature and have restored the Disaster Recovery backup instance from a set of backup files as described above, you must manually trigger replication of all media ingested since the

time that the backup files were created. Simplest is to trigger a manual replication from the primary instance to the backup instance for media ingested since the backup file set was created, **before (re)starting ingest** by using the "range" option of the start-replication command. Alternatively, you may instead use the start-replication "range" option, with "from" being the time the backup file set creation began, and the "through time" being the time the restore from those files was completed, and prior to any new media being ingested after the restore. See the [Disaster Recovery Management Console Commands](#) section for more information about the start-replication command.

Control Node Reconstruction

The Control Node (or nodes if the grid is configured with multiple, redundant Control Nodes as shown in the Large Distributed Deployment diagram in the <[Common Deployment Configurations](#)> section) contains a small database of data that is critical and needed by all other nodes. When there is only one Control Node, it is a single point of failure. If there is a catastrophic failure that causes loss of that data, it can be reconstructed from data queried from the gateway and ALL the data nodes. It should also be noted that if there is only one Control Node, when it is down, as well as during its reconstruction, Search Grid cannot process any data.

Steps for Control Node Reconstruction

Special Considerations When Grid Has Multiple Control Nodes:

- If the grid is configured with multiple, redundant Control Nodes, then the steps below *are generally not required* because the failed node will sync itself to the other nodes in the quorum once it is brought back online. **However, before bringing the failed node back online, you must completely delete the <data location>\control-node\control-data directory (and its content) before ever starting the Control Service.** This is even true if you have done a fresh installation of that control node on to a new server, because everything, including its inbox folder, must be gone, otherwise you risk losing data from the other control nodes instead of them populating the recovered one.
- If the control data is somehow bad or missing from *all* Control Nodes in a multiple Control Node grid, you can follow the instructions below, deleting control-data folders from *all* Control Nodes at the same time in step 1, below, but for step 2, set the property on only *one* of the Control Nodes, and it will be that on one that you examine the logs in steps 5 & 6.



1. Eliminate the failure cause:

- a. If there was a hardware failure, repair the hardware that failed, and if necessary, (re)install the Search Grid Control Node software, **but do not start the Search Grid Control Service yet.**
- b. If there was only control-data file corruption, then *with the Search Grid Control Service stopped*, completely delete the <data location>\control-node\control-data directory (and its content).

2. Edit the `<install location>\etc\control-node\control-service-properties.xml` file, and add the following line just **before** the final `</properties>` line:

```
<entry key="nexidia.searchgrid.control.zkReconstructionMode">enabled</entry>
```

3. Start the control-service
4. If the Gateway Service is down, or any Search Grid service is down on any of the data nodes, start those services.
The reconstruction will not complete until the Control Node is able to communicate with the Gateway node and ALL Data nodes.
5. Monitor the `<data location>\control-node\logs\control-service.log` file, watching for a log entry that states "ZOOKEEPER RECONSTRUCTION COMPLETE". (This should happen quickly once the gateway and ALL data nodes are running.)
6. **Perform all actions that are indicated as required in the control-service.log file.** All instructions will begin with "ACTION REQUIRED".



If reconstruction completed without any errors or warnings, meaning there was no conflicting or ambiguous data found on any of the data nodes, then all running services on all data nodes should have been automatically restarted, but you must still carefully read and follow all "ACTION REQUIRED" instructions in the log file.

Disaster Recovery

Disaster Recovery is a feature of Search Grid that, when enabled, automatically and regularly sends critical data to a designated Backup Search Grid instance, reducing data loss in the event of a catastrophic system failure of the Primary Search Grid instance and allowing for a relatively quick fail-over to the backup Search Grid. The Backup instance should ideally be located in a geographically separate data center in order to protect against catastrophic failures at one data center.

Connectivity between the Primary and Backup instance must be reliable, and the use of a VPN is recommended for added security. The exact topology is customer-specific; however, Search Grid Disaster Recovery does require specific connectivity between the Primary and Backup instances. **All** the Data Nodes in the Primary instance must have connectivity to the Backup instance's Gateway Node(s) and to **all** the Data Nodes of the Backup instance. Updates from the Primary instance to the Backup instance follow this path: Primary Data Node → Backup Gateway → Primary Data Node → Backup Data Node.

Critical data is sent from the Primary instance to the Backup instance in the form of a media bundle. Each media bundle contains a media record for a single media, segments for that media, and paths for that media, to be replicated from the Primary instance onto the Backup instance. All other data normally generated by Search Grid will be regenerated by the Backup instance once the bundle has been processed successfully.

Administrators can enable the Disaster Recovery feature to allow two identically configured Search Grid instances to be

paired together as Primary $\longleftrightarrow$ Backup.

For example, if the Primary instance has 1 Gateway Node, 1 Control Node, 3 Data Nodes, 3 Phrase Packs, 2 Language Packs, and 1 custom user (tenant), then:

- The Backup instance will also need to have those *exact same* node, packs and user configurations, with the processing power (CPU cores and speed), RAM size, and disk space & speed of the Backup instance matching (or exceeding) that of the Primary instance, *except the number of active gateways*. This not only ensures that the Backup instance will keep up with data it is being fed by the Primary instance and has all resources to properly process the data, but also ensures complete readiness and comparable performance in the event that the Primary instance fails and the Backup instance is put into full service.

It is CRITICAL that the backup grid instance has only one active gateway node. Multiple active gateways in the backup grid instance can prevent replication.

It is desirable, however, to have as many gateway services installed in the backup instance as there is in the primary instance so that they can all be enabled and used if there is ever a need to fail over the production environment, making the backup grid instance the primary instance.



If the primary grid instance has 3 gateways, then the backup grid should have 3 gateways *installed*, but with 2 of them **stopped & disabled**. Install the gateway services as normal, but afterward, ensure only one of the three gateway services is running, and that the other two are not only in the stopped state, but are also *disabled* to prevent accidentally starting them. Note that after every Search Grid upgrade, they will need to be disabled again.

A future update to Search Grid will add support for multiple backup gateways, but for now this warning must be heeded.

This restriction does *not* apply to the number of control nodes, **ONLY** to gateway nodes.

- All three of the Data nodes of the Primary instance must have network connectivity to the Gateway Node of the Backup instance and to *all three Data Nodes* of the Backup instance.

Disaster Recovery Data Flow

The following is an example data flow for a media to be replicated from the Primary instance to the Backup instance.

1. On the Primary instance:
 - a. Replication triggers on each Base service of Primary instance.
 - b. Base service determines all media currently owned by itself to be replicated.
 - c. Base service begins processing a media for replication.
 - d. Base service creates a media bundle.

- e. The media bundle is sent to the Backup instance's Gateway for media declaration.
2. On the Backup instance:
 - a. Gateway service receives the media bundle.
 - b. Gateway service determines if the media from the media bundle already exists on the Backup instance.
 - i. If the media does not exist on the Backup instance, it is declared, and a "Declared" response, which contains a link to the Base service on the Backup instance that owns that media, is sent to the Primary instance.
 - ii. If the media already exists on the Backup instance, an "Already Exists" response, which contains a link to the Base service on the Backup instance that owns that media, is sent to the Primary instance.
 3. On the Primary instance:
 - a. Base service receives the response from the Backup instance.
 - i. If the media does not have update data ready, processing on the next media begins. [Done with current media]
 - ii. If/when the media has update data ready to be replicated, Base service sends the media bundle directly to the Base service on the Backup instance that owns the media.
 4. On the Backup instance:
 - a. Base service receives the media bundle from the Primary instance.
 - b. Base service processes update data from bundle.
 - c. Base service sends "Update Successful" response back to Primary instance.
 5. On the Primary instance:
 - a. Base service receives "Update Successful" response from Backup instance. [Done with current media]
 - b. Base service begins processing next media.

Configuring Disaster Recovery

The Primary instance servers do not need any specific configuration during installation beyond the properties indicated below. The Primary instance's Data Node(s), however, must have network connectivity to the Backup instance's Gateway Node(s) *and* to **all** the Backup instance's Data Nodes. How that is achieved is customer-determined.

The Backup instance servers need one specific setting configured during installation to allow for connectivity from the

Primary instance. While running the installer for Search Grid, options for Configure Local Properties will appear. For these options, the "Public Bind Address" needs to be set to the *public IP address* of the server on the network — that is, the IP address of the network adapter that is on the network that has connectivity to the Primary instance's Data Nodes. The rest of the installer steps can be followed as normal. This can also be set after installation (but before enabling Disaster Recovery) by editing each Backup instance server's `<install location>\etc\local-properties.xml` file and setting the value for `nexidia.searchgrid.this.public.bindAddress`.



The public IP address can be found with the "ipconfig" command at a cmd prompt, and will be the address of one of the network adapters under the Ethernet adapter section. Consult the relevant IT department if unsure which is the correct address to use.



On the Backup instance, the admin password must match that of the primary instance. Tenants **must** be added, and language packs and phrase packs **must** be installed *before* enabling Disaster Recovery on the Backup and Primary instances. The data from the Primary being replicated to the Backup instance relies upon the Backup instance having the exact same Tenants, Phrase Pack(s) and Language Pack(s) as the Primary instance.

After Disaster Recovery is enabled, if there is ever a need to add or delete a Tenant or Phrase Pack, or to add a Language Pack, Disaster Recovery must be paused first. Once paused, make the changes to both the Primary and Backup instances before un-pausing.

Both the Primary instance and the Backup instance will need configurations set in the global properties by starting the *management-console*, logging in as "admin", and using the "set-gprop" commands as shown below:

Primary Instance Properties

```
# set-gprop nexidia.searchgrid.disasterrecovery.enabled true
# set-gprop nexidia.searchgrid.disasterrecovery.instanceRole primary
# set-gprop nexidia.searchgrid.disasterrecovery.paused false
# set-gprop nexidia.searchgrid.disasterrecovery.backupInstanceSystemUri http://[public_IP_address_of_backup_instance_gateway]:25002
```




For the `nexidia.searchgrid.disasterrecovery.backupInstanceSystemUri` setting, the value for `[public_IP_address_of_backup_instance_gateway]` needs to be the public IP address set during the Search Grid installation on the Backup instance.



It is **CRITICAL** that the backup instance has only one *active* gateway. See the warning near the beginning of the [Disaster Recovery](#) section for more details.

For example, if in the Backup instance's Gateway Node's `<install location>\etc\local-properties.xml` contains:

```
<entry key="nexidia.searchgrid.this.public.bindAddress">123.45.67.89</entry>
```

then the property would be set as follows:

```
# set-gprop nexidia.searchgrid.disasterrecovery.backupInstanceSystemUri http://123.45.67.89:25002
```



If you have configured the Backup instance to use HTTPS per the [HTTPS Support](#) section, then you must specify `https://` instead of `http://` for the `nexidia.searchgrid.disasterrecovery.backupInstanceSystemUri` property.

Optional Primary Instance Properties

The default values are shown below. **Do not set** unless there is a specific need to deviate from the default values:

```
# set-gprop nexidia.searchgrid.disasterrecovery.frequencyInSeconds 1
# set-gprop nexidia.searchgrid.disasterrecovery.backlogTriggerThreshold 1
```



All Search Grid services on all nodes of the Primary instance must be restarted after configuring an instance as the Primary instance.

Backup Instance Properties

```
# set-gprop nexidia.searchgrid.disasterrecovery.enabled true
```

```
# set-gprop nexidia.searchgrid.disasterrecovery.instanceRole backup
# set-gprop nexidia.searchgrid.disasterrecovery.paused false
```

See the [Disaster Recovery Global Properties](#) section for descriptions of each of the properties.



All Search Grid services on all nodes of the Backup instance must be restarted after configuring an instance as the Backup instance.

Disaster Recovery Automatic Replication

When the Disaster Recovery feature has been configured on a Search Grid instance, upon starting up, each Base service will make three checks:

1. Check if **nexidia.searchgrid.disasterrecovery.enabled** is set to 'true'
2. Check if **nexidia.searchgrid.disasterrecovery.instanceRole** is set to 'primary'
3. Check if **nexidia.searchgrid.disasterrecovery.paused** is set to 'false'

If all three of those conditions are met, replication will automatically begin on all Base services. Each Base service will check for new media ready to be replicated to the Backup instance every second, and will begin replication based on the values of:

- **nexidia.searchgrid.disasterrecovery.frequencyInSeconds** (default value: 1)
- **nexidia.searchgrid.disasterrecovery.backlogTriggerThreshold** (default value: 1)

When the number of "frequencyInSeconds" seconds has elapsed and media is ready for replication OR the number of media ready for replication reaches the "backlogTriggerThreshold" even if "frequencyInSeconds" has not elapsed, replication will begin.

Disaster Recovery Management Console Commands

The following *management-console* commands require logging into management-console as admin:

- If replication needs to be started manually, any the following three commands can be run to start the replication process with a specific range of media to be replicated:



The start-replication command is intended to be used only rarely, such as when first enabling the Disaster Recovery feature, or after recovering from some failure on the backup instance where it had to be restored from a backup and needs to be "caught up" with media ingested after backup's time.

The start-replication command can take significant time to complete. It completes after all the media for the requested timeframe has been enqueued for replication on all Data Nodes, at which time it will display that media count and indicates how long it took to enqueue them.



Depending upon the specified timeframe and the greatest number of media per node across all nodes that encompasses, **ingest may need to be stopped temporarily before issuing a start-replication command**, or ingest will encounter timeouts and will need to retry during the time that it takes to enqueue all the media for which replication was requested. Enqueuing takes roughly 30 seconds per million media (varying based upon server configuration/speed).

start-replication (hours|days|since|range) <value> [<dataNodeList>]

For **hours**, <value> is how many hours in the past from now.

For **days**, <value> is how many days in the past from now.

- This is equivalent to multiplying by 24 and specifying as hours.
-

For **since**, <value> is YYYY-MM-DD[@hh:mm[:ss[.nnn]]]

- If **@time** is omitted when running the command, it will use midnight on that date.
 - If **@time** is included when running the command, seconds and milliseconds are optional, and the time specified needs to be the local time for the server on which management-console is being run.
-

For range, <value> is a pair of date@time values, <from>~<through>

- <from> and <through> are formatted exactly as `since`, shown above, and are separated by a single tilde (~) character.
-
-

Optional [<dataNodeList>] allows replicating specified data nodes.

- Data node names must be separated by only a single comma.
 - Omission infers ALL data nodes.
-
-



If previous replication was done while one or more data nodes were down, after those nodes are back on-line, specifying just those previously-down node names allows replication to be performed again, but only on those nodes.

- If manually started replication needs to be cancelled for any reason, the following command can be used. It will *not* cancel media enqueued by automated, background replication:

```
# cancel-replication
```

- Disaster Recovery Metrics information, including Disaster Recovery configuration, replication success and failure rates, count of media currently awaiting replication, and oldest media awaiting replication, can be displayed with either of the following commands:

```
# get-disasterrecoverymetrics [<node-name>]
# get-drmet [<node-name>]
```

<node-name> is an optional parameter. If specified, must be the name of a Data Node.

- If <node-name> is omitted, it reports aggregated metrics from all Data Nodes.
 - If <node-name> is provided, it reports metrics for that single Data Node.
-

Testing Disaster Recovery Fail-Over

To *test* fail-over, then revert to the original primary instance, the following procedure **must** be used, and involves NIA as well as Search Grid. The scope of this document does not include any details about NIA, however high-level statements about NIA are mentioned in the procedure, below.

Search Grid allocates media IDs as media is declared. Because of how this works, one cannot simply switch NIA to the backup Search Grid instance, ingest media and test, and then revert NIA back to the primary instance and re-ingest that media. Doing so would cause conflicting IDs. In fact, there could already be IDs enqueued in the primary instance when it is paused to perform the test on the backup instance that would conflict with IDs allocated on the backup instance during the test.



To eliminate this conflict, the backup Search Grid instance must be put into a special test mode. While in this mode, media IDs get allocated with a very high value (2^{62} is added to the IDs). All other functionality of Search Grid is normal; only the media ID values are special. At the end of the test, NIA will drop that media from the backup instance, but physical deletion within Search Grid takes some time. Because those IDs are outside the range normally used, there will not be any conflicts prior to physical deletion if NIA is reverted to the primary Search Grid instance and the backup Search Grid instance is restored to backup-mode, resuming the processing of media replications sent from the primary instance.

For all management-console commands mentioned, user 'admin' is required.



Please refer to *Appendix K* in NIA's Installation Guide for details about the steps below that mention NIA.

The following procedure must be followed in this order:

1. Pause NIA and Prepare NIA for the DR test
2. Pause the *primary* Search Grid instance using the primary instance's management-console and entering the following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.paused true
```

3. Pause the *backup* Search Grid instance using the backup instance's management-console and entering the

following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.paused true
```

4. Disable Disaster Recover on the *backup* Search Grid instance using the backup instance's management-console and entering the following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.enabled false
```



Do not modify the 'instanceRole' setting. It must remain "backup" on the backup instance. Only 'paused' and 'enabled' are changed for this test mode.

5. Verify that the *backup* Search Grid instance is in Fail-Over Test Mode by using the backup instance's management-console and entering the following command:

```
# get-drmet
```

Under the Status heading, you should see:

```
Status
Enabled.....: false - In Backup Instance Fail-Over Test Mode
Role.....: backup
Replication Paused...: true
```

If not, do not proceed any further. Go back and check the settings modified above.

6. Restart all services in the *backup* Search Grid instance's Gateway service and the Base and Agent services on **ALL** data nodes.
7. Validate that all services restarted and have the updated configuration by using the backup instance's management-console and entering the following command:

```
# test-system
```

It should report "All tests passed". If not, rectify the issues it reports before proceeding.

8. Reconfigure NIA to connect to the *backup* Search Grid Instance
9. Ingest media from NIA and do whatever testing is desired.
10. **When testing is complete, have NIA drop all media it ingested for the test.**
11. Pause all transactions from NIA to the *backup* Search Grid instance.
12. Re-enable Disaster Recover on the *backup* Search Grid instance using the backup instance's management-console and entering the following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.enabled true
```

13. Verify that the *backup* Search Grid instance is no longer in Fail-Over Test Mode by using the backup instance's management-console and entering the following command:

```
# get-drmnet
```

Near the top of the output, under the Status heading, you should see:

```
Status
  Enabled.....: true
  Role.....: backup
  Replication Paused...: true
```

If not, do not proceed any further. Go back and check the 'enabled' setting modified above.

14. Restart the *backup* Search Grid instance's Gateway service and the Base and Agent services on **ALL** data nodes.
15. Validate that all services restarted and have the updated configuration by using the backup instance's management-console and entering the following command:

```
# test-system
```

It should report "All tests passed". If not, rectify the issues it reports before proceeding.

16. Un-pause the *backup* Search Grid instance using the backup instance's management-console and entering the following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.paused false
```

17. Un-pause the *primary* Search Grid instance using the primary instance's management-console and entering the following command:

```
# set-gprop nexidia.searchgrid.disasterrecovery.paused false
```

18. Reconfigure NIA to connect to the *primary* Search Grid Instance
19. Return NIA to normal operations on the *primary* Search Grid Instance.

Disaster Recovery Fail-Over Procedure

In the scenario where the Primary instance has suffered a catastrophic failure, the following set of procedures should be followed to use the Backup instance while the Primary instance is being restored.

1. Start the *management-console* on the Backup instance, log in as "admin", then enter the following commands exactly as below to remove the global properties:

```
# set-gprop nexidia.searchgrid.disasterrecovery.enabled
# set-gprop nexidia.searchgrid.disasterrecovery.instanceRole
# set-gprop nexidia.searchgrid.disasterrecovery.paused
```

2. Stop and re-start **all** Search Grid services on **all** nodes of the Backup instance.

At this point, the Backup instance is ready to be used as the main Search Grid instance while the Primary instance's servers are restored. All external services that use Search Grid must be redirected to the Backup instance.



All structured queries **must** be re-ingested to the Backup instance. **Some** media that was sent to Search Grid from before the point in time when the Primary instance went down, **must** be re-ingested to the Backup instance. *How much to media to re-ingest* depends upon the volume of data a site ingests and the replication latency. A general guideline is to re-ingest 2x the average replication latency the system typically has. For example, if average latency is 15 minutes, then re-ingest the last 30 minutes of media that were ingested prior to the primary instance going down.

Since latency is part of the Disaster Recovery metrics of a functioning Primary instance during replication, it is good to become familiar with the site-specific latency before a fail-over is needed by occasionally checking and recording the latency as reported by the *management-console* "get-drmet" command.

Disaster Recovery Fail-Back Procedure

There are two basic paths for fail-back:

- **CLEAN Option:** Starting with a completely clean installation of Search Grid on the former Primary instance servers, then configuring the former Primary instance as the Backup instance, configuring the former Backup instance as the Primary instance, and then finally initiating replication from the new Primary instance to the new Backup instance. This is more of a "starting anew" option than it is a literal "fail-back".
 - **Advantages:** This is the simplest approach. It requires only a very minimal down-time for Search Grid during which ingestion must be stopped.
 - **Disadvantages:** The amount of time to replicate to the clean instance can be long, depending on the number of media involved. Until replication is caught up, there effectively is no backup to fail over to if there is a failure of the Primary (formerly Backup) instance. Also, (not necessarily a disadvantage, but a difference) the Primary and Backup instances remain reversed.
- **RESTORE Option:** Depending on the type of failure, this option may or may not require a clean installation of Search Grid, but it will require a grid-consistent snapshot of the running (former Backup) instance. This means that Search Grid must be stopped during the time all control and nodes are being backed up.
 - **Advantages:** Likely, the restored system will be fully functional and caught-up sooner. It is also possible for the Primary and Backup instance back to be flipped their original Search Grid instances prior to the failure.
 - **Disadvantages:** There could be significant down-time required to get a grid-consistent backup of all nodes. Depending upon the geographical distance between two data centers, there could be additional time in transferring the backup to the other data center.

Regardless of the fail-back option chosen, the failure of the original Primary instance must be resolved. Once the failure state has been resolved, depending on the type of and extent of the failure and/or the fail-back option chosen, a clean installation of Search Grid may need to be put onto the Primary instance server, including reinstalling any phrase packs, language packs, and creating all necessary tenants.

CLEAN Option steps:

1. Start with a fresh installation of Search Grid and fresh installations of language and phrase packs on all servers of the failed instance. If the failure left data remaining, delete it. It is critical to start completely fresh. Do not forget to add any custom tenants and synchronize tenant and admin passwords between the running instance and the newly-rebuilt instance.
2. Configure the newly-rebuilt instance as a Disaster Recovery Backup instance, then restart all Search Grid services on all nodes of the Backup instance. Please refer to the [Backup Instance Properties](#) section for details.
3. Configure the currently running (former Backup) instance as a Disaster Recovery Primary instance, then restart all Search Grid services on all nodes of the Primary instance. Please refer to the [Primary Instance Properties](#) section for details.
4. Use the management-console "start-replication" command to initiate replicating already-ingested media from the Primary instance to the Backup instance. Please refer to the [Disaster Recovery Management Console Commands](#) section, paying attention to the warning about possibly needing to briefly pause ingest.
 - To minimize replication time, do not replicate media that is older than needed in the case of a failure scenario.
 - During the replication of old media, replication of media newly ingested on the Primary instance will be processed after the old media has been replicated. (Replication order is based upon media's ingestion time.)

RESTORE Option steps:

1. Repair the Primary instance's server(s) that failed, and if necessary, freshly install Search Grid on the failed server(s).
2. Stop all Search Grid services on all nodes of BOTH instances.
3. Create backup copies of all nodes on the Backup instance. Please refer to the [Backups](#) section, and do an offline backup, ensuring ownership and permission attributes are retained in the backup (for example, by using robocopy with the /COPYALL option or xcopy with the /O option). Note that this backup must be grid-consistent, meaning the entire grid must be stopped while all nodes are backed up.
4. If the goal is make the original Primary instance the Primary instance again, skip to the next step. ONLY if the goal is to make the Backup instance the Primary instance and make the original Primary Instance the new Backup instance, then...
 - a. Start *only* the Control service on the control node(s) of the former Backup instance.

- b. Start `management-console` and enter primitive mode (press the enter key when prompted for a user name).
 - c. Configure Search Grid as a Primary instance **EXCEPT**, for the `nexidia.searchgrid.disasterrecovery.paused` setting, set it to **true** (not to false). Please refer to the [Primary Instance Properties](#) section for details.
 - d. Restart all Search Grid services on all nodes of the *new Primary* instance. (Do not start the services of the repaired instance yet.)
5. Restore the backups taken from *all* Backup instance nodes to *all* nodes of the repaired instance, ensuring ownership and permission attributes are copied from the backup.
6. ONLY if the goal is to make the original Backup instance the Primary Instance and make the original Primary Instance the new Backup instance, then...
 - a. Start *only* the Control service on the control node(s) of the former Primary instance.
 - b. Start the management-console and enter primitive mode (press the enter key when prompted for a user name).
 - c. Configure Search Grid as a Backup instance. Please refer to the [Backup Instance Properties](#) section for details.
 - d. Restart all Search Grid services on all nodes of the *new Backup* instance.
 - e. On the new Primary instance, start `management-console`, login as admin, then enter the following command to un-pause replication:

```
# set-gprop nexidia.searchgrid.disasterrecovery.paused false
```
 - f. You are done. It will take time for the media ingested and enqueued while paused was 'true' in the new Primary instance to replicate to the new Backup instance and become caught up. How much time it will take depends on how much media was ingested during that period.
7. ONLY if the goal is to make the repaired, original Primary instance is the Primary instance again, then...
 - a. **Stop all ingest.**
 - b. On the Backup instance, start `management-console` and configure that instance as a backup instance again, then restart all Search Grid services on all nodes of the Backup instance. Please refer to the [Backup Instance Properties](#) section for details.
 - c. Start all Search Grid services on all nodes of the repaired Primary instance.
 - d. On the repaired Primary instance, start `management-console` and configure that instance as a primary instance. Please refer to the [Primary Instance Properties](#) section.

- e. **RE-ingest** all media that had been ingested since the backup had been created. You may want to re-ingest from a little further back in case you're not positive of the exact point ingest was stopped to create the backup.
- f. Allow normal ingest to resume.
- g. You are done.

TCP Port Usage

This section lists default port assignments to support firewall configuration. Grid servers must allow incoming connections on the ports listed below. All ports are used on the grid-private network interface except where noted otherwise. See the [Configuration Property Reference](#) section for information on how to change the port and network interface settings.

Service	Port	Purpose
Gateway	25002	HTTP (public)
	25102	HTTP
	25103	JMX
	25130	Control/Election
Control	25132	HTTP
	25133	JMX
	25134	Control/Connection
	25135	Control/Quorum
Base	25121	Database
	25221	Database (node-private)

Service	Port	Purpose
	25022	HTTP (public)
	25122	HTTP
	25222	HTTP (node-private)
	25123	JMX
	25142	HTTP
Agent	25143	JMX
Compute	25112	HTTP
	25113	JMX

Moving Search Grid's Folders

There may be occasions where it becomes necessary to move Search Grid data after installation. For example, an additional, larger and/or faster disk was installed, and it is desired to move Search Grid's data to that disk. **There are steps that must be taken to guarantee success and allow future upgrades to succeed.**



You must NOT use Windows Explorer to copy or move the data. **Use a file copy utility that preserves file ownership and permission attributes.** (The xcopy command can be run with the /O option to preserve file ownership and permission attributes; the /COPYALL flag performs a similar function for robocopy.)

Moving Search Grid's Installation Location

- You must never move the installation location without uninstalling and then re-installing Search Grid. Moving only the folders can cause the installer (.msi) to fail the next time an upgrade is attempted. Do not worry; uninstalling Search Grid does not delete any data — just be certain to have the Search Grid installer available for re-installation. Please, carefully read through all the steps before beginning.

1. Use Windows' "Add or Remove Programs" to uninstall Search Grid. (Do not delete *anything* manually.)

2. Run the Search Grid installer, installing all the same components that were previously installed.
3. Where prompted with the "Install location" and "Data Location" paths near the bottom of the installer's dialog box, edit **ONLY** the "Install Location" path to the desired new location.



Do NOT modify the Data Location. It *must* be the same path as before. If this gets changed, Search Grid will create new, empty folders in the new location, but we need it to use the location of the existing data. (Moving the data location is more complex. See [Moving Search Grid's Data Location](#), below.)

4. When the installation is complete, before starting any Search Grid services, you must copy the `etc` subdirectory from the previous installation location to the new installation location, overwriting files in the new installation location with the files from the previous installation location, **being certain to preserve file ownership and permission attributes** as mentioned above.
5. Start all Search Grid services and verify everything is functioning properly. Check each service's log files for WARN level and ERROR level messages *before* ingesting any new media.
6. After Search Grid is verified to be functioning properly, you may delete the old installation folder if desired.

Moving Search Grid's Data Location

When preparing to move Search Grid's data location, be sure to allow ample down-time to finish. All Search Grid services must be stopped, and the *entire* move must be done without restarting them. (Copying some of the data one day, restarting Search Grid, stopping it, and moving the rest of the files later is **invalid** and *will result in corrupt data and/or search grid crashes*.) It is also recommended that you copy, not move, the data, so it's simpler to fall back to the old location if something goes wrong during the transfer. Please, carefully read through all the steps before beginning.



These instructions presume that any Search Grid client (NIA, for example) has been stopped/paused before beginning, and does not get restarted/resumed until after completing all steps.

1. Stop all Search Grid services.
2. Copy everything from the original data location to the new data location, **being certain to preserve file ownership and permission attributes** as mentioned above.
3. Update the `nexidia.searchgrid.node.data.path` property for *every Search Grid node type installed on the server*. The property files are located below the `<install location>\etc` folder (usually `c:\Program Files\Nexidia\Search Grid X.Y`, where X.Y is the major.minor version, such as 20.1), in the subdirectories named for each of the node types installed on the server (compute-node, control-node, data-node, gateway-node) *and* if it exists, apps:

```
<install location>\etc\apps\apps-properties.xml
<install location>\etc\compute-node\compute-service-properties.xml
<install location>\etc\control-node\control-service-properties.xml
<install location>\etc\data-node\data-node-properties.xml
```

| <install location>\etc\gateway-node\gateway-service-properties.xml

- For each of those files that exist on the server, you will need to edit them using a text editor (notepad.exe is fine), but depending on the server's security policy and your user's permissions, you may need to run the editor "as administrator" in order to save the files after editing them.



It is recommended that you make a backup copy of the entire `etc` folder before making changes.

The required changes for each file that exists are as follows:

- Find the entry for `nexidia.searchgrid.node.data.path`
- Note the current path value, which is between a `>` and a `<`. *For example*, the `apps-properties.xml` file might contain:

| <entry key="nexidia.searchgrid.node.data.path">C:\ProgramData\Nexidia\Search Grid\apps</entry>

- Change the path to the new location, being careful to (a) retain the final subdirectory (`\apps` in the above example) and (b) to not change anything else. *For example*, if your new Search Grid Data location is `G:\SearchGridData`, change it to:

| <entry key="nexidia.searchgrid.node.data.path">G:\SearchGridData\apps</entry>

- Save your changes.
 - Repeat for each of the above 5 files that exist on the server.
- Update the Search Grid entries in the Windows Registry. Failure to do this will guarantee problems the next time Search Grid gets upgraded, especially if `NxDeployer` is used, which relies upon the registry entries to know where to look for Search Grid data. Since that could be months or years later, it is best to update the registry immediately after moving Search Grid's data, so the task does not become forgotten. To edit the Windows Registry, follow these simple steps:

- Run `regedit.exe`
- Navigate in `regedit` to:

| Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Nexidia, Inc.\Nexidia Search Grid\X.Y (where X.Y is the major.minor version, such as 20.1)

- In the right-hand pane, double-click on the `dataDirPath` item.
- Modify the "Value data:" field to reflect the new path to the Search Grid Data. If the previous path included a trailing backslash (`\`), include a trailing backslash on the new path.
- Click on OK to save the new value.

- f. Navigate in regedit to:

Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Nexidia, Inc.\Nexidia Search Grid\
MostRecentInstallation

- g. In the right-hand pane, double-click on the `dataDirPath` item.
 - h. Modify the "Value data:" field to reflect the new path to the Search Grid Data. If the previous path included a trailing backslash (\), include a trailing backslash on the new path.
 - i. Click on OK to save the new value.
 - j. Close the regedit app.
5. Rename the old data location folder so Search Grid cannot use it, in case any of the properties files modifications were missed.
 6. Start all Search Grid services and verify everything is functioning properly *before* ingesting any new media.
 - a. Check each service's log files for WARN level and ERROR level messages.
 - b. Verify that Search Grid did not recreate the old data location folder.
 - c. Verify that Search Grid did not create any new folders in the new data location folder (for example, if there was a typo in the modified properties files).
 7. After Search Grid is verified to be functioning properly, you may delete the old data location folder if desired.

Configuration

Log File Configuration

Search Grid uses the Logback framework for configuring log files. Each service's log configuration file is stored in `<install location>\etc\<node type>\<service name>-log-config.xml`. The most common reason for editing the log configuration files is to change the rolling policy of log files.

By default, services retain a set of up to 10 log files with a maximum size of 10 MiB each, plus the active log file, also with a maximum size of 10 MiB. The sizes and number of files can be changed if needed. The name and location of the active log file for each service is `<data location>\<node type>\<service name>.log`, with the retained logs being `<data location>\<node type>\<service name>.<sequence number>.log`, where a sequence number of 1 is the most recent log prior to the active log file. The framework can also be configured to roll and retain log files based upon time windows rather than size. See the [Logback website](#) for documentation on the format of the log configuration files.

Agent, Base, Compute and Gateway services also have a second, separate set of log files for performance logging. The logging frequency and volume of performance-related logs are typically much less than that of a services' normal logs. These log files roll based upon time windows (months). By default, up to 12 months of performance logging history is retained, with a maximum total retained size of 120 MB (plus the currently active log), allowing for the examination of performance trends over a long timeframe. Like the regular log files, the retention policy for the performance log files can be changed if necessary, as can the frequency at which the performance metrics get logged; however, it is recommended to leave these settings at their defaults unless actively diagnosing a performance issue. NOTE: The performance metrics will also get logged into the normal service log files, but they roll more frequently due to all the other logging.)

Each service also has a startup log file, into which the service's environment and configuration is written when the service starts. That file is not appended to; it is overwritten at each service start-up, and therefore there will only ever be one, small startup log file per service.



Changes to the `<service name>-log-config.xml` files will take effect within 10 seconds of saving the file; therefore it is *unnecessary* to restart services after making logging configuration changes.

Configuration Files

Configuration of individual servers, nodes, and services is set through configuration files. Search Grid configuration files use the Java XML property file format. An example property entry is shown below; for additional details on the file format, see the Java API documentation.

```
<entry key="property.name">property value</entry>
```

Global Configuration Management

Global configuration properties are stored in a special replicated data store in the Control Nodes, and edited through one of two mechanisms: Management Console and the Control Node inbox. Management Console is intended for interactive use or scripted use (see [Management Console Scripting](#)) while the Control Service is running, whereas the Control Node inbox is intended to support automated installation when the Control Service is not running.

Global Configuration via Management Console

Unlike most Management Console functions, the only prerequisite for viewing and editing global properties is that a quorum of Control Services be running. No other Search Grid services are required; therefore, global properties can be managed in Management Console's Primitive Mode.

Management Console provides two ways to edit global properties. The simplest way is to use the `set-globalproperty` command to control properties individually. Another option is to import and export global properties from and to a file.

Exporting global properties to a file should be used with caution. Consider a Search Grid deployment with three Control

Nodes. An administrator exports the global properties to a file on server A, edits it, then imports it to Search Grid. Later, another administrator edits a global property. At this point, the exported file on server A no longer represents the state of the global configuration. If the first administrator edits the file and imports it again, the changes made by the second administrator will be overwritten.

To avoid the unintentional overwrite scenario, exported global property files are "stamped" with a property that uniquely identifies the state of the global properties when they were exported. If any global property is changed before the file is imported, Search Grid rejects the import, preventing the changes from being overwritten. If this happens, export the file again to get the most recent settings. Alternatively, if the intention is to overwrite the settings, delete the "nexidia.searchgrid.properties.exportedRevision" property from the exported file.

Note that most global property changes do not automatically apply to running services, as described in the next section.

Management Console Commands

The following commands are available in Admin and Primitive Modes.

- To set a single global property:

```
# set-globalproperty nexidia.searchgrid.propertyName value
```

- To un-set / remove a single global property, do not specify a value:

```
# set-globalproperty nexidia.searchgrid.propertyName
```

- To display all global properties:

```
# get-globalproperties
```

- To export all global properties to a file:

```
# get-globalproperties <output filename>
```

- To import an existing global property file:

```
# set-globalproperties <input filename>
```

Control Node Inbox Support

To submit global configuration properties to the Control Node inbox, write a file called "global-properties.xml" using the same Java XML properties format used in most Search Grid configuration files. For example, the file must contain the standard XML headers and properties tag, as such:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
<properties version="1.0">
  <!-- Add property entry items here, between these begin and end properties tags. -->
```

| </properties>

Configuration properties injected from an inbox file **replace all** pre-existing global properties. Any properties not present in the global-properties.xml file will be unset.



For more details about inbox use, particularly when there are multiple Control Nodes, see the [Control Node Inbox](#) section.

Applying Configuration Changes

Most configuration property changes—in files or global properties—do not automatically apply to running services. Applying configuration changes requires a logical restart of the affected services. This can be cumbersome for a large deployment, especially for global properties that affect many services. To make this easier, Management Console allows an administrator to logically restart Search Grid services, thereby applying all configuration changes.

When the restarts are complete, the test-system command will indicate that all tests pass.

Note that restarting services may interrupt availability to client applications.

Management Console Commands

The following commands are available in Admin and Primitive Modes.

- To restart the minimum set of Search Grid services affected by configuration changes:

```
| # restart-system minimal
```

- To restart all Search Grid services regardless of configuration change:

```
| # restart-system full
```

The **restart-system** command is broadcast to all Search Grid nodes, but there is no confirmation that it was heard by every node. Therefore:



- After using the **restart-system minimal** command, it is recommended to use **test-system** in Admin mode to verify that all nodes that had been missing configuration changes no longer report that error.
- After using the **restart-system full** command, it is recommended to use **test-system -verbose:Start** in Admin mode and examine the details in the response to verify that all services, except for Control services, indicate they have indeed restarted.

Configuration Property Reference

This section describes configuration properties, what they control, and where they are located. Each property, together with its default value, is listed in the appropriate format. Properties set in files are shown in XML format, and global properties are shown as they would be entered in Management Console. This makes it easy to cut and paste from this document into the appropriate application.

Each property is described as being required or not. If a property is not required and is not present in the configuration file, Search Grid will use the property's default value. If a property is required, it must be explicitly defined. The initial values of some required properties are set at installation time.

Properties that accept an IP address will accept either IPv4 or IPv6 addresses.

The following tokens are used in many property descriptions in this section:

- `[data location]` This token refers to the directory where all grid nodes installed on a given server will store data files, including phonetic indexes. Usually this defaults to `C:\ProgramData\Nexidia\Search Grid`, but a different value may be chosen at install time. After installation, the value of the data location may be set via properties described in this reference.
- `[node data location]` This token refers to the directory where a specific node will store its data files. By default, this is a node-specific subdirectory of `[data location]`. After installation, the value of the node data location may be set via properties described in this reference.

Any property, whether mentioned in the [Global Properties](#) section or the various [File-Based Configuration](#) sections, can be set as a global property in order to share that value with all Search Grid services on all node types.

If the same property is set in multiple locations, the location set that is the most specific for a service is the value that will be used, so **care must be taken when choosing where to set a property**. Rarely, if ever, does it make sense to set a property in more than one location.

The order of precedence (if a property is set in multiple locations), from lowest-to-highest is:



1. global
2. server (local-properties.xml)
3. data-node-properties.xml (only applicable to data nodes)
4. property file named for a service (for example, base-service-properties.xml)

Global Properties

Global properties are stored in Control Nodes, and edited using Management Console (see [Global Configuration Management](#) section).



Unless otherwise stated for a particular property, after updating a global configuration property, ALL Search Grid services on ALL nodes must be restarted for the new value to take effect properly.

The following global configuration properties are available and should always be set as a global property, *never* in a property file:



Default values are shown next to the property names, or [no default value] is shown for those that have no default value.

nexidia.searchgrid.lss.enabled true

Required: No

Controls whether LSS should be enabled on this Search Grid instance. If LSS is enabled, it is usable only with language packs for which a phonetic digest model is installed. All 8.0 and later language packs include a phonetic digest model, but the phonetic digest model must be installed separately for 7.x language packs.

Set to "false" to conserve the CPU, IO, and storage expense of supporting LSS.

nexidia.searchgrid.nodeNameList [no default value]

Required: Yes

Sets a comma-delimited list of Data Node names in use. Leading and trailing whitespace is trimmed. The node names must match the values of `nexidia.searchgrid.node.name` in the `data-node-properties.xml` file of each Data Node. The following example shows a set of three Data Nodes. It could be used in the global properties inbox:

```
<entry key="nexidia.searchgrid.nodeNameList">node1, node2, node3</entry>
```

nexidia.searchgrid.mediadeclaration.highAvailabilityEnabled false

Required: No

Controls the media declaration availability mode. The default value, false, sets normal availability mode. Set to true to enable high availability mode.

nexidia.searchgrid.languageidmodel.fileName 2ref_NAEnglish_CanFrench_LASpanish.ALS

Required: No

Sets the filename of the language identification model to use. The default demonstration model is a good choice for North American content, especially for telephony. The default model enables the following language ID values in language identification requests:

Language	ID
English, North American	0
French, Canadian	31
Spanish, Latin American	1

To use a demonstration model supporting several European languages, set the property value to "2ref_English_French_Spanish_German_Italian_Broadcast.ALS". This model supports the following language ID values:

Language	ID
English, UK	9
French, European	29
German	8
Italian	33
Spanish, Castilian	30

If you have acquired a different language identification model from NiCE, or if you have created your own model with Nexidia AudioFinder, configure the system as follows:

- Copy the model file to each server hosting a Data Node or a Compute Node in `<install location>\share`. Note that this step must be performed on *all* servers hosting a Data Node or a Compute Node.
 - Set the above property to the model's filename. The property value must include the extension (usually .ALS) but not include any leading path information.
-

nexidia.searchgrid.lockoutpolicy.enabled true

Required: No

Controls whether users are locked out based on lockout policy criteria. If set to false, users are never locked out.

nexidia.searchgrid.lockoutpolicy.lockoutThresholdNum 3

Required: No

Sets the number of authentication failures that must occur during the lockout window to trigger lockout.

nexidia.searchgrid.lockoutpolicy.lockoutWindowMins 1

Required: No

Sets the length of time in minutes during which authentication failures count toward lockout.

nexidia.searchgrid.lockoutpolicy.lockoutDurationMins 5

Required: No

Sets the length of time in minutes that a user remains locked out. Negative values are treated as infinite, so user lockout will be permanent unless unlocked via other means, e.g. Management Console.

nexidia.searchgrid.cancelTenantsMediaOps [no default value]

Required: No

A comma-delimited list of tenants whose media operations should be administratively canceled. See [Administratively Canceling Media Operations](#) section for details.

Disaster Recovery Global Properties



See the [Configuring Disaster Recovery](#) section for detailed instructions on configuration of the Disaster Recovery feature.

Properties Applicable to Primary and Backup Instances

nexidia.searchgrid.disasterrecovery.enabled false

Required: No

Controls if the Disaster Recovery feature is turned on for the current server.

nexidia.searchgrid.disasterrecovery.instanceRole [No default value]

Required: No

Controls the server instance role for the Disaster Recovery feature. Accepted values are "primary" and "backup" and must be lowercase.

nexidia.searchgrid.disasterrecovery.paused false

Required: No

Controls if the Disaster Recovery process is currently paused.

When this property is changed, it takes effect immediately. Services do NOT need to be restarted.

If this property is being set to true in order to enable management-console commands that are disabled while Disaster Recovery is active (for example set-password or add-languagepack):



- It should be set to true on both the primary and backup instances.
- The same changes should be performed on both instances.
- When done, it should be set to false on both instances.

Properties Applicable to Primary Instance Only

nexidia.searchgrid.disasterrecovery.backupInstanceSystemUri [No default value]

Required: No

Sets the public addresses of the instance gateway for the Disaster Recovery feature. Value must be "http://<IP Address>:<port>". The IP Address must match the nexidia.searchgrid.this.public.bindAddress of the configured backup instance gateway server. Port must match the nexidia.searchgrid.gateway.webservice.public.port of the backup gateway server, if set to a custom value that server, otherwise the default port of 25002 must be used.

nexidia.searchgrid.disasterrecovery.frequencyInSeconds 1

Required: No

Controls how often the Disaster Recovery automated backup process runs on the Primary server.

nexidia.searchgrid.disasterrecovery.backlogTriggerThreshold 1

Required: No

Controls the maximum number of media that can be queued up at once before replication will be triggered earlier than the next scheduled run.

File-Based Configuration

There are configuration files that apply an entire server (all Search Grid services on that server), to an entire data node (both Agent and Base services on that data node), and to individual Search Grid services of any node type.



Default values are shown below, if applicable, between the closing angle-bracket (>) of the entry tag, and the opening angle-bracket (<) of the /entry tag. If those brackets are together with nothing between them (><), then there is no default value for that property.

Server Configuration

The <install location>\etc\local-properties.xml file contains properties that apply to all nodes and Search Grid applications installed on the server. When updating a common configuration property, the services on that server must be restarted for the new value to take effect.

<entry key="nexidia.searchgrid.this.public.bindAddress">127.0.0.1</entry>

Required: Yes

Sets the local server's NIC on which services will open server sockets for communication with client applications. On the server hosting the Gateway Node, this is the address on which the Gateway Service will listen for requests to the System URI (see Programmer's Guide for more information).

<entry key="nexidia.searchgrid.this.gridprivate.bindAddress">127.0.0.1</entry>

Required: Yes

Sets the local server's NIC on which services will open server sockets for communication with other grid services. For optimum performance, this should be set to an isolated subnet dedicated to inter-node communication; however, if the server has only a single NIC, this value should be the same as the value for `nexidia.searchgrid.this.public.bindAddress`.

<entry key="nexidia.searchgrid.control.gridprivate.port">25134</entry>

Required: No

Sets the port that local control service binds to / listens on. If a custom value is set for this port number, that same port number *must* be specified in the `nexidia.searchgrid.control.connectionString` property, below.



If Search Grid is configured with multiple Control Nodes, each is allowed to have its own custom port number so long as the `nexidia.searchgrid.control.connectionString` property, below, contains the matching port numbers for each control node.

<entry key="nexidia.searchgrid.control.connectionString">127.0.0.1</entry>

Required: Yes

Sets the addresses that local grid services use to find the Control Services. This setting should have the same value on all Search Grid services. The format is a comma-delimited list of grid-private IP addresses; or, if custom ports are in use, of <IP Address>:<port> pairs. Each IP address must match the value of `nexidia.searchgrid.this.gridprivate.bindAddress` on a Control Node server. Although this property may be set to a hostname, it is recommended to use a numeric IP address to avoid confusion when installing on servers with multiple NICs. IPv6 address must be enclosed in square brackets [], and if specifying custom ports for IPv6 addresses, the :<port> goes after the closing bracket.

This example shows a system with three Control Nodes using default port numbers:

```
<entry key="nexidia.searchgrid.control.connectionString">10.0.1.7,10.0.1.8,10.0.1.9</entry>
```

This example shows the same system with custom ports (refer to `nexidia.searchgrid.control.gridprivate.port` setting, above):

```
<entry key="nexidia.searchgrid.control.connectionString">10.0.1.7:26134,10.0.1.8:26134,10.0.1.9:27134</entry>
```

This example shows use of an IPv6 address:

```
<entry key="nexidia.searchgrid.control.connectionString">[2001:db8:85a3:0:0:8a2e:370:7334]</entry>
```

This example shows use of an IPv6 address with a custom port:

```
<entry key="nexidia.searchgrid.control.connectionString">[2001:db8:85a3:0:0:8a2e:370:7334]:26134</entry>
```

```
<entry key="nexidia.searchgrid.control.quorum.gridprivate.port">25135</entry>
```

```
<entry key="nexidia.searchgrid.control.election.gridprivate.port">25130</entry>
```

Required: No

These two properties set the port number used by the Control Service to communicate with itself, and with other Control Nodes if so configured, to determine if a quorum of Control Services are up, and to determine the leader. These ports are used even if Search Grid is configured for a single Control Node (quorum of 1, it's its own leader) or if configured for multiple Control Nodes (quorum is a majority, its leader gets elected).



If there are multiple Control Nodes, they communicate with each other using these ports, and therefore, unlike `nexidia.searchgrid.control.gridprivate.port` it is **critical** that, if setting custom port numbers for either or both of these properties, ALL Control Nodes are configured to use the same custom port numbers in their own respective `local-properties.xml`.

```
<entry key="nexidia.searchgrid.https.enabled">false</entry>
<entry key="nexidia.searchgrid.https.keystore.location">[no default value]</entry>
<entry key="nexidia.searchgrid.https.keystore.password">[no default value]</entry>
<entry key="nexidia.searchgrid.https.keystore.certificateAlias">searchgrid</entry>
<entry key="nexidia.searchgrid.https.clientsAuthenticateServers">false</entry>
```

Required: No

See the [HTTPS Support](#) section for a description of these properties.

```
<entry key="nexidia.searchgrid.jmx.enabled">true</entry>
```

Required: No

Controls whether to allow diagnostic tools to connect to the services on the grid-private network. This is not necessary for normal functioning, but it can greatly facilitate engagements with NiCE technical support personnel.



Please note that even if enabled, the JMX ports will not be opened unless the file "enable.jmx" is created in the `<install location>\bin` directory. That file is normally not present, and as such, setting this property to false serves only to prevent starting JMX when `<install location>\bin\enable.jmx` exists. It is therefore recommended to leave this property unset (defaulting to true), allowing NiCE technical support to use JMX to debug issues on-site by simply creating `enable.jmx` if needed, without having to restart any services to see changes to this property's value.

Gateway Node

Gateway Service

The `<install location>\etc\gateway-node\gateway-service-properties.xml` file contains properties that apply to the Gateway Service. The Gateway Service must be restarted for the changes to take effect.

<entry key="nexidia.searchgrid.node.data.path">[data location]\gateway-node</entry>

Required: Yes

Sets the location where the node's data files will be stored. The value of this property is used to derive the default values of several other properties. In this document, default values with the token [node data location] refers to the value of this property.

<entry key="nexidia.searchgrid.languagemodelservice.searchPath"></entry>

Required: No

Sets the directory or directories to scan for automatic language pack installation. Separate multiple directories with a semicolon. Setting to a blank value (the default) disables automatic language pack installation. See the [Automatic Language Pack Installation](#) section for more information.

<entry key="nexidia.searchgrid.searchAudit.level">OFF</entry>

Required: Yes

Controls whether the Gateway Services keeps a separate log of all search activity. The default value, OFF, disables the search activity log. Set to INFO to enable the search activity log for searches. Set to DEBUG to log autothreshold activity as well as searches.

<entry key="nexidia.searchgrid.searchdirector.searchExpirationTimeSeconds">60</entry>

Required: No

Controls how frequently a client application must invoke Get Search Info for a particular search. If a client fails to do so, the search will be canceled. This value may need to be raised in servers that experience heavy CPU or network IO load spikes. To enforce a consistent policy across all Gateway Nodes, this property can be set in the global properties.

<entry key="nexidia.searchgrid.gateway.webservice.public.port">25002</entry>

Required: No

Sets the server port that client applications will use to communicate with the Gateway Service. This value is the port used in the System URI (see Programmer's Guide for more information).

<entry key="nexidia.searchgrid.gateway.webservice.gridprivate.port">25102</entry>

Required: No

Sets a port that grid nodes use to communicate with the Gateway Service.

<entry key="nexidia.searchgrid.jmx.gateway.gridprivate.port">25103</entry>

Required: No

Sets the port on the grid-private network over which diagnostic tools can communicate with the Gateway Service.

<entry key="nexidia.searchgrid.webservice.storagePath">[node data location]\web-server</entry>

Required: No

Sets a location to write temporary files that support communication between nodes. At least 1 MiB should be free on this volume.

<entry key="nexidia.searchgrid.database.storagePath">[node data location]\database</entry>

Required: No

Sets the location in which to create the Gateway Node's database. At least 2 GiB should be free on this volume.

<entry key="nexidia.searchgrid.storage.model.path">[node data location]\phonetic-data\models</entry>

Required: No

Sets the location to which the Gateway Service copies installed phonetic models it finds. This location must have enough space to hold all installed language models, usually under 100 MiB per language pack.

<entry key="nexidia.searchgrid.log.dir">[node data location]\logs</entry>

Required: No

Sets the location where the Gateway Service writes support log files.

Data Node

All Data Node Services

The `<install location>\etc\data-node\data-node-properties.xml` file contains properties that apply to all Search Grid services in the Data Node. All Data Node services must be restarted for changes to this file to take effect.

<entry key="nexidia.searchgrid.node.name">[no default value]</entry>

Required: Yes

Sets the name of this Data Node. This name must also appear in the `nexidia.searchgrid.nodeNameList` property in the global properties. This property has no default value, and must be updated after the Data Node is installed.

<entry key="nexidia.searchgrid.node.data.path">[data location]\data-node</entry>

Required: Yes

Sets the location where the node's data files will be stored. The value of this property is used to derive the default values of several other properties. In this document, default values with the token [node data location] refers to the value of this property.

<entry key="nexidia.searchgrid.node.capacity">1.0</entry>

Required: No

Sets the share of media to be assigned to this Data Node during media declaration and data redistribution. Increase or decrease the value for this Data Node to receive a larger or smaller share of media. In general, the value should be proportional to the storage capacity of this node relative to other Data Nodes in the grid. So if three Data Nodes each have 500 GiB of storage for phonetic indexes and another Data Node has 750 GiB, the former nodes should have a capacity value of 1.0; the latter node, 1.5. If decommissioning a node, set the value to 0.0.

<entry key="nexidia.searchgrid.storage.pat.path">[node data location]\phonetic-data</entry>

Required: No

Sets the location where pat files should be stored on this Data Node.

<entry key="nexidia.searchgrid.storage.pat.minimumFreeSpaceGb">3.0</entry>

Required: No

Sets the minimum amount of free space required on the pat storage volume. If the volume has less than this amount of free space, new pats will not be created or accepted for upload.

<entry key="nexidia.searchgrid.storage.monogest.path">[node data location]\phonetic-data</entry>

Required: No

Sets the location where monogest files should be stored on this Data Node. Monogest files consume 2.5 MiB per hour of media.

<entry key="nexidia.searchgrid.storage.monogest.minimumFreeSpaceGb">2.0</entry>

Required: No

Sets the minimum amount of free space required on the monogest storage volume. If the volume has less than this amount of free space, new monogests will not be created.

<entry key="nexidia.searchgrid.storage.polygest.path">[node data location]\phonetic-data</entry>

Required: No

Sets the location where polygest files should be stored on this Data Node. Polygest files consume 2.5 MiB per hour of media.

<entry key="nexidia.searchgrid.storage.polygest.minimumFreeSpaceGb">1.0</entry>

Required: No

Sets the minimum amount of free space required on the polygest storage volume. If the volume has less than this amount of free space, new polygests will not be created.

<entry key="nexidia.searchgrid.storage.archive.in.path">[node data location]\phonetic-data</entry>

Required: No

Sets a location for writing temporary files during data redistribution.

<entry key="nexidia.searchgrid.storage.archive.in.minimumFreeSpaceGb">1.0</entry>

Required: No

Sets the minimum amount of free space required on the archive storage volume. If the volume has less than this amount of free space, data redistribution cannot progress on this Data Node.

<entry key="nexidia.searchgrid.storage.archive.out.path">[node data location]\phonetic-data</entry>

Required: No

Sets a location for writing temporary files during data redistribution.

<entry key="nexidia.searchgrid.log.dir">[node data location]\logs</entry>

Required: No

Sets the location where the Data Node services write support log files.

<entry key="nexidia.searchgrid.node.database.nodeprivate.port">25221</entry>

Required: No

Sets a port on the node-private network used for communication between services in this Data Node.

<entry key="nexidia.searchgrid.node.webservice.nodeprivate.port">25222</entry>

Required: No

Sets a port on the node-private network used for communication between services in this Data Node.

Base Service

The `<install location>\etc\data-node\base-service-properties.xml` file contains properties that

apply to the Data Node's Base Service. The Base Service must be restarted for changes to this file to take effect.

<entry key="nexidia.searchgrid.webservice.storagePath">[node data location]\web-server\base</entry>

Required: No

Sets a location to write temporary files that support communication between nodes. At least 1 MiB should be free on this volume.

<entry key="nexidia.searchgrid.database.storagePath">[node data location]\database</entry>

Required: No

Sets the location for the Data Node's database. At least 64 KB per media hour to be stored on this Data Node should be free on this volume.

<entry key="nexidia.searchgrid.polygestcreation.maxMonogestAgeMinutes">60</entry>

Required: No

Controls the longest amount of time a monogest will be allowed to remain unpolygested and therefore unsearchable via LSS. Usually, polygest creation is triggered when enough monogests accumulate in a Data Node to achieve a target polygest size, but if the target size cannot be achieved by the time specified by this property, the Base Service will create a new polygest regardless of its size. Because small polygests are less efficient to search, the service will ultimately merge multiple small polygests to create larger ones. Therefore, reducing this value reduces this time it takes to make media searchable at the expense of IO and CPU load.

To enforce a consistent policy across all Data Nodes, this property can be set in the global properties.

<entry key="nexidia.searchgrid.physicaldeletes.mediaOperationHoursBeforeDelete">1</entry>

Required: No

Sets the length of time that media operation info is retained for a completed media operation request. After this time, the Get Media Operation Info operation for this request will return a 404 (Not Found) response. (See Programmer's Guide for more information.) Note that this property affects only the media operation info, not the notification. To enforce a consistent policy across all Data Nodes, this property can be set in the global properties.

<entry key="nexidia.searchgrid.webservice.base.public.port">25022</entry>

Required: No

Sets the port on the public network used for communication with client applications.

<entry key="nexidia.searchgrid.webservice.base.gridprivate.port">25122</entry>

Required: No

Sets a port on the grid-private network used for communication between nodes.

<entry key="nexidia.searchgrid.database.base.gridprivate.port">25121</entry>

Required: No

Sets a port on the grid-private network used for communication between nodes.

<entry key="nexidia.searchgrid.jmx.base.gridprivate.port">25123</entry>

Required: No

Sets the port on the grid-private network over which diagnostic tools can communicate with the Base Service.

Agent Service

The `<install location>\etc\data-node\agent-service-properties.xml` file contains properties that apply to the Data Node's Agent Service. The Agent Service must be restarted for changes to this file to take effect.

`<entry key="nexidia.searchgrid.webservice.storagePath">[node data location]\web-server\agent</entry>`

Required: No

Sets a location to write temporary files that support communication between nodes. At least 1 MiB should be free on this volume.

`<entry key="nexidia.searchgrid.patcache.enabled">true</entry>`

Required: No

Controls whether pats should be cached in memory to improve conventional search performance. The pat cache can be particularly effective for "drill down" searches, in which the search space comprises the media resulting from the previous search. However, the pat cache incurs its own CPU and memory overhead, so applications should test whether it actually provides a performance benefit. To enforce a consistent policy across all Agent Services, this property can be set in the global properties.

`<entry key="nexidia.searchgrid.patcache.memoryTargetGb"></entry>`

`<entry key="nexidia.searchgrid.patcache.memoryTargetPhysicalMemoryPercent"></entry>`

Required: No

Controls the maximum process memory size to target if the pat cache is enabled. The `nexidia.searchgrid.patcache.memoryTargetGb` property sets the target size to an absolute value, while `nexidia.searchgrid.patcache.memoryTargetPhysicalMemoryPercent` calculates the target size as a function of the physical RAM available on the server. Setting both properties to a non-blank value results in an error. If both properties are left unset, the target size is the smallest of 50% of total physical RAM or 2 GiB. To enforce a consistent policy across all Agent Services, this property can be set in the global properties.

`<entry key="nexidia.searchgrid.search.threadCount">-1</entry>`

`<entry key="nexidia.searchgrid.search.OverthreadCount">-1</entry>`

Required: No

Controls the number of threads used for search and compute operations in the Agent Service. If `nexidia.searchgrid.search.threadCount` is set to -1, the thread count will equal the number of logical CPU cores plus the value of `nexidia.searchgrid.search.OverthreadCount`. If `nexidia.searchgrid.search.threadCount` is set to a positive value, the value of `nexidia.searchgrid.search.OverthreadCount` is ignored. To enforce a consistent policy across all Agent Services, this property can be set in the global properties.

```
<entry key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.pre10.8">2</entry>
<entry
key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.smallTransformer">0</entry>
<entry
key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.largeTransformer">0</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.custom">0</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.diarizationBeingUsed">true</entry>
```

Required: Yes, if default values do not match site's phrase pack use.

See [Configuring Phrase Recognition](#) section for more information.

```
<entry key="nexidia.searchgrid.gpu.usage">try</entry>
<entry key="nexidia.searchgrid.phraserecognition.serverReserveMemoryGb">0.0</entry>
<entry key="nexidia.searchgrid.phraserecognition.dataNodeOverheadGb">10.0</entry>
<entry key="nexidia.searchgrid.phraserecognition.ratioOfMemory">0.9</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.maxMemoryGb"></entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.maxConcurrentOperations"></entry>
```

Required: No

See [Configuring Phrase Recognition](#) section for more information.

```
<entry key="nexidia.searchgrid.storage.tmpmedia.dir">[node data location]\temp-media</entry>
```

Required: No

See the [Compute Service](#) section for a description of this property.

<entry key="nexidia.searchgrid.compute.enableOnAgent">true</entry>

Required: No

Controls whether the Agent Service performs compute operations. Setting this to false can reduce latency of search requests when the system is under load, especially in an environment that processes long-duration media files. However, a setting of false also underutilizes CPU resources on the Data Node. A false value should be used only on a deployment that includes one or more Compute Nodes; otherwise, the system cannot perform media operations or digesting. To enforce a consistent policy across all Agent Services, this property can be set in the global properties.

<entry key="nexidia.searchgrid.webservice.agent.gridprivate.port">25142</entry>

Required: No

Sets a port on the grid-private network used for communication between nodes.

<entry key="nexidia.searchgrid.jmx.agent.gridprivate.port">25143</entry>

Required: No

Sets the port on the grid-private network over which diagnostic tools can communicate with the Agent Service.

<entry key="nexidia.searchgrid.database.gridprivate.enabled">true</entry>

Required: No

Controls whether the grid-private database service is enabled. It is intended for use only in deployment environments where policy forbids network database connections.

Compute Node

Compute Nodes



Compute Nodes, when used, MUST be on their own servers. A Compute Node must never be installed on the same server that is running any other type of node, especially a data node. Data Nodes and Compute nodes both presume to be able to consume nearly all RAM that is available when the service (Agent service on a Data Node or Compute service on a Compute Node) starts. If a Compute Node is installed along with any other node type on the same server, memory could be over-allocated, causing Search Grid, or even the operating system, to crash.

Compute Service

The `<install location>\etc\compute-node\compute-service-properties.xml` file contains properties that apply to the Compute Service. The Compute Service must be restarted for changes to this file to take effect.

`<entry key="nexidia.searchgrid.node.data.path">[data location]\compute-node</entry>`

Required: Yes

Sets the location where the node's data files will be stored. The value of this property is used to derive the default values of several other properties. In this document, default values with the token [node data location] refers to the value of this property.

`<entry key="nexidia.searchgrid.compute.threadCount">-1</entry>`

`<entry key="nexidia.searchgrid.compute.overthreadCount">1</entry>`

Required: No

Controls the number of threads used for compute operations. If `nexidia.searchgrid.compute.threadCount` is set to -1, the thread count will equal the number of logical CPU cores plus the value of `nexidia.searchgrid.compute.overthreadCount`. If `nexidia.searchgrid.compute.threadCount` is set to a positive value, the value of `nexidia.searchgrid.compute.overthreadCount` is ignored. To enforce a consistent policy across all Compute Services, this property can be set in the global properties.

`<entry key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.pre10.8">2</entry>`

`<entry`

```
key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.smallTransformer">0</entry>
<entry
key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.largeTransformer">0</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.custom">0</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.diarizationBeingUsed">true</entry>
```

Required: Yes, if default values do not match site's phrase pack use.

```
<entry key="nexidia.searchgrid.gpu.usage">try</entry>
<entry key="nexidia.searchgrid.phraserecognition.serverReserveMemoryGb">0.0</entry>
<entry key="nexidia.searchgrid.phraserecognition.computeNodeOverheadGb">9.5</entry>
<entry key="nexidia.searchgrid.phraserecognition.ratioOfMemory">0.9</entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.maxMemoryGb"></entry>
<entry key="nexidia.searchgrid.compute.phraserecognition.maxConcurrentOperations"></entry>
```

Required: No

See [Configuring Phrase Recognition](#) section for more information.

```
<entry key="nexidia.searchgrid.storage.tmpmedia.dir">[node data location]\temp-media</entry>
```

Required: No

Sets the location to write temporary media files downloaded from web servers. This directory is used only if client applications make media available on web servers rather than the file system. Files are stored in this location only long enough to complete a media operation, but if the service is terminated, media files may be left in place until the service is restarted. Note that on startup, the service deletes any files found in this directory.

```
<entry key="nexidia.searchgrid.webservice.storagePath">[node data location]\web-server\compute</entry>
```

Required: No

Sets a location to write temporary files that support communication between nodes. At least 1 MiB should be free on this volume.

<entry key="nexidia.searchgrid.log.dir">[node data location]\logs</entry>

Required: No

Sets the location where the Compute Service writes support log files.

<entry key="nexidia.searchgrid.webservice.compute.gridprivate.port">25112</entry>

Required: No

Sets a port on the grid-private network used for communication between nodes.

<entry key="nexidia.searchgrid.jmx.compute.gridprivate.port">25113</entry>

Required: No

Sets the port on all NICs over which diagnostic tools can communicate with the Compute Service.

Configuring Phrase Recognition

Hyper-Threading should be *disabled* on all nodes performing phrase recognition, whether Compute Nodes or Data Nodes, *unless a GPU is being used*. With Hyper-Threading enabled, the logical core count doubles, but it effectively reduces the per-core cache sizes, and phrase recognition is very math-centric, CPU-intensive *and* RAM-intensive. Because of this, *unless a GPU is being used*, Hyper-Threading adds at most very little benefit, and often may slightly *degrade* phrase recognition performance, even though it may look like there are more threads doing the work. Those extra threads also require a lot of additional RAM, which can affect overall server performance.



- Without a GPU, optimal phrase recognition performance is achieved with Hyper-Threading disabled and when it is using all physical cores (assuming enough RAM is installed to allow it to use them all) and when the configuration properties below are properly set.
- If *exclusively* using a GPU (see the [restrictToGpuIfPresent](#) property below), optimal phrase recognition performance is achieved with Hyper-Threading enabled, if the configuration properties below are properly set.

- If using a GPU, but *also* allowing the CPU to be used (see the [restrictToGpuIfPresent property](#) below), phrase recognition Hyper-Threading may or may not benefit phrase recognition performance, depending on the balance of how much computational work the GPU is doing vs. the CPU. This may vary by the hardware configuration, as well as how the configuration properties below are set.

Please also see the cautionary notes in the **Search Grid System Requirements Guide** regarding virtual memory and NUMA settings.

GPU acceleration is available for acceleration of phrase recognition and pat creation for Language Pack versions 8.7 and greater.

<entry key="nexidia.searchgrid.gpu.usage">try</entry>

This value applies only when a compatible GPU is present. It this setting controls how a GPU is used to accelerate phrase recognition and pat creation. The default value, "try" will use a GPU if one is detected. If a GPU is observed to cause problems, a value of "none" will prevent it from being used. A value of "force" will cause the service to terminate if a usable GPU is not found. This can be helpful in test situations where it is not certain that a given GPU is compatible or configured correctly. To enforce a consistent policy across all Agent and Compute Services, this property can be set in the global properties.



Older GPUs that were specified for Search Grid 3.1 WILL NOT WORK for Search Grid 12.4 and later, and while the small and large transformer models are much more accurate, phrase recognition performance using the transformer models can be 3x-10x slower without GPU support. Please refer to the latest Search Grid System Requirements document for the supported GPU configurations.

Phrase recognition consumes more CPU and memory resources than other Search Grid functions, so tuning a Search Grid system for phrase recognition deserves special consideration. Loading each phrase pack requires about 1.5 to 2.25 GiB of RAM, and each phrase recognition operation can consume roughly 0.75 GiB to 1.2 GiB more while running. In addition, diarization (commonly used along with phrase recognition) consumes up to 500 MiB per operation. To control memory consumption, Data and Compute Nodes limit how many phrase recognition operations run in parallel based on the amount of available physical RAM in the server, the phrase recognition model type, and if diarization is being used. It must, however, determine this when the Agent and Compute services start-up and therefore must be told, via configuration settings, what type of work to expect in the worst-case scenario. How much phrase recognition work the CPU is allowed to do while the GPU is being used can also be controlled.

<entry key="nexidia.searchgrid.phraserecognition.restrictToGpulfPresent">true</entry>

This value applies only when a compatible GPU is present. (See the NiCE Search Grid System Requirements document for GPU compatibility.) This setting determines how much load can be placed on the CPU when the GPU is being used for phrase recognition. The CPU will always have some load, but most of the computational work can be done by the GPU. Depending upon how much system RAM exists in the server vs. the GPUs, the server may be able to do some phrase recognition work on its own, in parallel with the GPU. How efficiently the CPU can be used also depends on the phrase pack model(s) being used. **In general, however, if using the Small Transformer or Large Transformer phrase packs, restricting to the GPU gives the best performance if using the GPUs stated in the Search Grid System Requirements document.** Pre-10.8 phrase packs *may or may not benefit* from setting this to false, depending on server hardware configuration and GPU type. If hardware configurations are identical for all Agent and Compute Services, adjustments away from the default for this property may be set in the global properties via the *management-console* "set-gprop" command.

If set to false *and* there is enough system RAM in the server that would allow more parallel phrase recognition than would be allowed on the GPU alone, work may overflow to the CPU. For example, if the GPU can accommodate 20 concurrent operations, and the CPU's RAM can accommodate 10 more, then 20 operations would be done concurrently on GPU, plus 10 more on the CPU. **However, if set to false, and the CPU isn't powerful enough, it could slow performance.** It is made available for sites that have GPUs installed in sufficiently powerful servers for the phrase pack versions they're using.

With default settings, Data and Compute Nodes conservatively determine the amount of physical RAM allowed to be used for phrase recognition. Therefore, manual adjustments to the properties may improve performance. Each of the following three (3) settings should be examined, and if needed, adjusted from their defaults to site-specific values. To enforce a consistent policy across all Agent and Compute Services, adjustments away from the defaults for these three properties should be set in the global properties via the *management-console* "set-gprop" command. Setting the proper values are important to most efficiently using RAM without exhausting it completely. Default values are shown.



The properties `nexidia.searchgrid.compute.phraserecognition.phrasePacksInUse` and `nexidia.searchgrid.compute.phraserecognition.modelType` have been deprecated and MUST NOT be set. If set, the Agent and Compute services will fail to start.

`nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.pre10.8` 2

`nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.smallTransformer` 0

`nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.largeTransformer` 0

These values must be set to match the number of Phrase Packs *being used* at a site. Note that this may be less than the number of phrase packs installed if some installed packs are no longer being used. Typically, this is one per language, but may be higher.

If there are multiple tenants, each of these values must be set to *the sum of **unique** phrase packs of each type from all tenants*. If an identical phrase pack (same GUID) is used by multiple tenants, that should only be counted once.



For example, if one tenant has four pre10.8 phrase packs in use (A, B, C, and D), and another tenant has two pre10.8 phrase packs in use (A and E) and one smallTransformer phrase pack in use, then `phrasePacksInUseCount.pre10.8` must be set to 5, and `phrasePacksInUseCount.smallTransformer` must be set to 1.

There is also a set of properties to allow for future, customer-specific, customized phrase recognition models, which may have different memory requirements from the standard models, above:

`nexidia.searchgrid.compute.phraserecognition.phrasePacksInUseCount.custom 0`

This value should be set *only if a customized model is provided*, and if this value is set, then the following two values must also be set to valid values for the customized model or Search Grid may fail upon start-up:

`nexidia.searchgrid.compute.phraserecognition.modelSizeGb.custom`
`nexidia.searchgrid.compute.phraserecognition.operationSizeGb.custom`

These two values are used only when `phrasePacksInUseCount.custom`, above, is set to a value > 0. If using a customized model, then these values must be set to the exact values provided by NiCE along with the customized-model Phrase Pack.

`nexidia.searchgrid.compute.phraserecognition.modelSizeGb.custom.gpu`
`nexidia.searchgrid.compute.phraserecognition.operationSizeGb.custom.gpu`

These two values are used only when `phrasePacksInUseCount.custom`, above, is set to a value > 0 and a GPU is being used. If using a customized model, then these values must be set to the exact values provided by NiCE along with the customized-model Phrase Pack.

nexidia.searchgrid.compute.phraserecognition.diarizationBeingUsed true

In simple terms, diarization is the process used to distinguish between multiple voices (caller & customer service agent) within a single (monaural) audio track. If a site's audio files are all stereo, diarization probably is not being used and this property can be set to **false** in order to allow more memory to be used to provide more concurrent phrase recognition operations, improving performance.

<entry key="nexidia.searchgrid.phraserecognition.serverReserveMemoryGb">0.0</entry>

<entry key="nexidia.searchgrid.phraserecognition.ratioOfMemory">0.9</entry>

<entry key="nexidia.searchgrid.phraserecognition.dataNodeOverheadGb">10.0</entry>

<entry key="nexidia.searchgrid.phraserecognition.computeNodeOverheadGb">9.5</entry>

These four items go hand-in-hand and are each floating-point (aka "real") numbers. The default value of `dataNodeOverheadGb` (applies only to Data Nodes) and `computeNodeOverheadGb` (applies only to Compute Nodes) are conservative values determined by R&D and should suffice in most deployments without risking RAM exhaustion. If RAM usage is too high, for example if some customer-specific application such as antivirus software needs RAM on a server, or if RAM is being underutilized *and* there are sufficient non-Hyper-Threading cores to efficiently utilize more RAM for improved performance, then these values can be adjusted, carefully, *in small increments*, to ensure RAM usage is maximized without risking exhaustion. Extreme care should be taken to not set values that run too close to memory exhaustion because though RAM usage may look good if observed over a short time, phrase recognition of some media files may require more RAM, if even briefly, and enough headroom must be available to accommodate them.

Note that setting `serverReserveMemoryGb` to a negative value effectively reduces `dataNodeOverheadGb` (and `computeNodeOverheadGb` on compute nodes) by the amount specified, and effectively increases them if set to a positive value. The reason `serverReserveMemoryGb` is provided separately is primarily to accommodate changes to a specific server without adjusting the R&D-determined `dataNodeOverheadGb` default value, but it could be applied to all, globally, with proper consideration.

- If an adjustment needs to be made only for specific servers, then the most logical item to change would be `serverReserveMemoryGb` in the `local-properties.xml` OR the `data-node-properties.xml` on the specific server(s).
- If a value needs to be changed equally for all Data Nodes OR all Compute Nodes, then the most logical item to change would be to set `dataNodeOverheadGb` and/or `computeNodeOverheadGb` in the global properties using the (*management-console* `set-gprop` command). For example,

```
# set-gprop nexidia.searchgrid.phraserecognition.computeNodeOverheadGb 9.5
```

- If the value needs to be changed equally for all Data Nodes **and** all Compute Nodes, then `serverReserveMemoryGb` could optionally be set in the global properties, but remain aware that when `serverReserveMemoryGb` is set globally, it affects Data Nodes **and** Compute Nodes.

The `ratioOfMemory` property is a floating-point value that sets the max amount of available memory to use as a percentage of memory *after all other reductions* such as from `serverReserveMemoryGb` and the `*NodeOverheadGb` values. A value of 1.0 should work, but by default a safety margin is provided to better protect against out-of-memory conditions. The value must be *greater* than 0.0 and *less than or equal to* 1.0.



The above `nexidia.searchgrid.phraserecognition` properties should be the primary method used to adjust values for performance-tuning a Search Grid installation.

Alternatively, the amount of memory for phrase recognition can be explicitly controlled with the following setting in Agent and Compute Service config files:

```
<entry key="nexidia.searchgrid.compute.phraserecognition.maxMemoryGb"></entry>
```

- The default value (empty) allows Search Grid to calculate the limit based upon all the other `nexidia.searchgrid.phraserecognition` properties listed above, starting with the amount of physical RAM available when the service starts (less a small safety threshold).
- When set, however, the lesser of this value and the total amount of physical RAM available at service start-up (less the small safety threshold) will be used *with no additional calculations based upon the other `nexidia.searchgrid.phraserecognition` properties.*

When using this setting, it is extremely important to consider other consumers of memory on the server, such as the Agent Service's pat cache, other Search Grid services, other applications, and the operating system itself, including how

much the other consumers' memory usage can grow over time. If overall memory consumption grows large enough to cause paging to disk, system performance will be dramatically reduced, or if the paging file's size is exceeded (or if it is disabled), the server could crash without warning. Though changing values always warrants careful consideration, it is generally safer to adjust the other `nexidia.searchgrid.phraserecognition` properties above. Note: If this value is set higher than the low memory warning threshold when the Agent of Compute service starts, the low memory warning threshold will be used. (The low memory warning threshold is determined by the amount of RAM available when the service starts, *minus* a small amount of headroom that is determined based upon the amount of *total* RAM installed.)

Instead of controlling phrase recognition concurrency indirectly via memory limits, it is also possible to control concurrency directly with the following setting, also in Agent and Compute Service config files:

```
<entry key="nexidia.searchgrid.compute.phraserecognition.maxConcurrentOperations"></entry>
```

This is the riskiest setting to use. The default value (empty) allows Search Grid to calculate the limit. If this property is set to a value larger than the number of search threads (see `search.threadCount` and `search.overthreadCount` under [Agent Service](#)) or compute threads (see `compute.threadCount` and `compute.overthreadCount` under [Compute Service](#)), the smaller value will take precedence, however the amount of RAM installed in the server is not taken into consideration at all. Therefore, it is very easy to cause server crashes due to resource (memory) exhaustion by setting this value too high. If this setting is used, it is best used as a means to set a *lower* value for `maxConcurrentOperations` than would have otherwise been calculated based upon memory, not to set a higher value, though if done with care, that will work, too.



It is invalid to set a non-empty value for both the `maxMemoryGb` and `maxConcurrentOperations` on the same server. The Agent or Compute Service will terminate on startup if so configured.

Restricting to Compute Nodes

A common scenario in systems using phrase recognition without ad hoc search is to deploy many Compute Nodes and one (or a few) Data Nodes. In such deployments, it can make sense to prevent the Data Nodes from locally performing phrase recognition, isolating high CPU and memory requirements to the Compute Nodes. To accomplish this, place the following line in the Agent Service config file of each Data Node:

```
<entry key="nexidia.searchgrid.compute.workCapabilities">MEDIA_OP,MONOGEST,GPU,NON_GPU</entry>
```

Whitelisting Media Locations

Beginning with Search Grid 3.1.27, a new security feature exists, but it is opt-in (disabled by default). This feature allows whitelisting media file locations to prevent using file URIs passed over the network from accessing locations other than where media files are stored. The whitelist applies only to direct file access (not http access, which would be up to the http server to restrict).



This feature is intended to augment, not replace, user-level access restrictions. User-level access restrictions should be the primary means for restricting access for the Search Grid services. This whitelist is intended to

add further protection from any malicious activity on the network attempting to gain access to files elsewhere, but accessible via the user-level restrictions.



When this feature is enabled (the whitelist is not empty), the whitelist must include ALL allowed media locations. Operations passing URIs to locations other than what are whitelisted *will fail*, so make sure when creating the whitelist that all media file locations are included.

This new property is: `nexidia.searchgrid.mediaLocationsWhitelist`



Please note that the property name, like all property names, is case-sensitive.

- If all Data and Compute nodes access a single repository for media files, then this whitelist can be set via the global properties in Search Grid's *management-console* using the following command, where the format of 'paths' is shown below in the Example section:

```
set-globalproperty nexidia.searchgrid.mediaLocationsWhitelist paths
```

- If, in a multi-node grid, specific Data and/or Compute nodes access media files in locations specific to that node, then this whitelist can be set via the local-properties.xml file on each specific server **OR** by the data-node-properties.xml file on each specific server. (The format of 'paths' is shown below in the Example section.)

```
<entry key="nexidia.searchgrid.mediaLocationsWhitelist">paths</entry>
```

- Multiple locations can be specified by separating them with a semicolon.
- Specifying a location in the whitelist permits *all locations below that* in the directory structure, so the list can be as specific or as broad as desired. This allows each site to decide the granularity of control over whitelisted location – wide, specifying only top-level directories, or narrow, specifying every specific directory.

Example:

The Windows paths will look just like paths used to access a windows share on a Windows desktop PC:

```
\\server-a\share-x\path\to\media;\\server-a\share-x\path\to\media\subdir;\\server-a\share-x\path\to\moremedia;\\server-a\share-y\path\to\othermedia
```

This would whitelist the following locations:

```
\\server-a\share-x\path\to\media (and all subdirectories)
\\server-a\share-x\path\to\moremedia (and all subdirectories)
\\server-a\share-y\path\to\othermedia (and all subdirectories)
```

Note that the `\\server-a\share-x\path\to\media\subdir` entry in the whitelist is redundant because it falls below `\\server-a\share-x\path\to\media`.

Common Exception Protocol (CEP) Server Configuration

Search Grid can be enabled to connect to NiCE's CEP Server product. This allows Search Grid to send periodic heartbeats that the CEP server can monitor, as well as sending any error (or in some cases warning) log messages for the CEP server to store and analyze. All messages sent to the CEP server are encrypted during transport.

By default, CEP is disabled in Search Grid, but can be enabled using either the default self-signed certificate that the CEP Server creates for clients when it is installed, or a custom (customer-provided) certificate.



To use a custom certificate, that certificate will also have to be installed on the CEP server. This document only covers configuring CEP within Search Grid and installing certificates into Search Grid. Please refer to the CEP Server documentation for its certificate installation instructions.

For more information about certificates and keystores, you can refer to the [A Little About Certificates](#) and [A Little About Keystores](#) sections. Those sections are related to enabling HTTPS support, but they're generic enough to gain an understanding from.



Some of the information below about installing certificates is redundant with or similar to instructions in the [External Keystore Security](#) sub-section of HTTPS Support. That section includes screen captures which may be helpful, so even if HTTPS is not being used for Search Grid in general, it may be advantageous to read those sections about importing custom certificate before following the process below for CEP.

Using The Default CEP Client Certificate

The simplest and quickest way to get CEP up and running in Search Grid is to use the certificate that the CEP server created when it was installed. Consult the CEP Server documentation for the location of that certificate, but it will be named `artemis.cer`.



This process needs to be done on every server that is running Search Grid services.

1. Copy the certificate file to anywhere on the Search Grid server. The rest of these steps are all performed on the Search Grid server.
2. Using Windows search in the task bar, type `mmc`, and then choose the "Run as administrator" option.
3. Once the MMC UI opens, in the top menu, click on `File`, then choose `Add/Remove Snap-in...`
4. When the Add or Remove Snap-ins dialog opens, click on the `Certificates` item at the left, then click on `Add >`.
5. When prompted for the type of account, choose `Computer account` then click `Next`.
6. When prompted for the computer to manage, choose `Local computer (the computer this console is running on)` then click `Finish`.

7. You should now see the Add or Remove Snap-ins dialog again. Click [OK](#) to return back to the main MMC window.
8. At the left, click on [Certificates](#).
9. In the center pane will be a list of folders. RIGHT-click on the [Trusted Root Certification Authority](#) folder and then choose [All Tasks](#) then [Import](#). This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to [Local Machine](#), then click [Next](#).
11. Click on the [Browse...](#) button. This will open an "Open" file dialog.
12. Browse to and locate the artemis.cer file you copied from the CEP server.
13. Select the artemis.cer file, then click [Open](#).
14. You will see a dialog that provides some information about the certificate. Click on [Next](#).
15. Verify that the [Place all certificates in the following store](#) item is selected, and that the [Certificate store:](#) value is [Trusted Root Certification Authority](#), then click [Next](#).
16. Click [Finish](#).
17. It should pop up a message saying "The import was successful." Click [OK](#).
18. You should now be back at the main MMC window. In the center pane, DOUBLE-click on the [Trusted Root Certification Authorities](#) folder.
19. In the center pane, DOUBLE-click on the [Certificates](#) folder. The center pane should fill with a long list of certificates.
20. Find the certificate in the list that is named the same as the CEP server's name.
21. RIGHT-click on that certificate and choose [Properties](#).
22. On the [General](#) tab, near the top, in the [Friendly Name](#) field, enter [CEP-Artemis](#) then click [OK](#).
23. In the list of certificates, for that certificate, you should now see CEP-Artemis under the Friendly Name column. You may now close MMC. If it asks to save the Console settings, you may choose [No](#).

Using Custom CEP Certificates

Importing Custom Trusted Root Certification Authority Certificates

Follow the steps under [Using The Default CEP Client Certificate](#) except specify your custom certificate instead of artemis.cer. If your custom certificate is within a keystore file, at step 12, where you are browsing to the certificate, set

the file type (to the right of the [File Name](#) field) to "All Files (.*)" in order to see your keystore, and at step 13, select your keystore file and when you click on [Open](#) you will be prompted to enter the password to your keystore file.

Importing a Dual-Use Certificate

If you have a single, custom certificate that is to be used for both encryption and authenticating the CEP server, see [Importing Custom Trusted Root Certification Authority Certificates](#), above, to install that single certificate.

Importing Encryption Certificates

If using separate certificates for authentication and encryption, import the custom certificate to be used for authentication-only following the steps in [Importing Custom Trusted Root Certification Authority Certificates](#), above, and then install the certificate that is to be used for encryption-only following these steps:

1. Copy the encryption-only certificate file to anywhere on each Search Grid server, or if that certificate is within a keystore, copy that keystore file to anywhere on each Search Grid server. The rest of these steps are all performed on each Search Grid server.
2. Using Windows search in the task bar, type `mmc`, and then choose the "Open file location" option. This will open a Windows Explorer window, usually in the `C:\Windows\System32` directory, and will highlight `mmc.exe`
3. While holding the SHIFT key, RIGHT-click on `mmc.exe`, then choose the "Run as different user" option.
4. When prompted for user credentials, enter the Search Grid *service user's* credentials.
5. Once the MMC UI opens, in the top menu, click on [File](#), then choose [Add/Remove Snap-in...](#)
6. When the Add or Remove Snap-ins dialog opens, click on the [Certificates](#) item at the left, then click on [Add >](#).
7. Click [OK](#) to return back to the main MMC window.
8. At the left, click on [Certificates](#).
9. In the center pane will be a list of folders. RIGHT-click on the [Personal](#) folder and then choose [All Tasks](#) then [Import](#). This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to [Current User](#), then click [Next](#).
11. Click on the [Browse...](#) button. This will open an "Open" file dialog.
12. Click on the drop-down to the right of the [File Name](#) field, and choose "All Files (.*)"
13. Browse to and locate the custom certificate file or the keystore file containing the trusted certificate that file you copied from the CEP server.

14. Select that file, then click [Open](#).
15. You will see a dialog that provides some information about the certificate. Click on [Next](#).
16. Verify that the [Place all certificates in the following store](#) item is selected, and that the [Certificate store: value is Trusted Root Certification Authority](#), then click [Next](#).
17. You will be prompted to enter the keystore's password. Enter the password in the [Password](#) field, verify that the [Include all extended properties](#) option has a check-mark, then click [Next](#).
18. Verify that the option selected is [Place all certificates in the following store](#) and the [Certificate Store](#) field contains "Personal", then click [Next](#).
19. Click [Finish](#).
20. It should pop up a message saying "The import was successful." Click [OK](#).
21. You should now be back at the main MMC window. In the center pane, DOUBLE-click on the [Personal](#) folder.
22. In the center pane, DOUBLE-click on the [Certificates](#) folder. The center pane should fill list of personal certificates for the service user. (What you added may be the only one.)
23. Find the certificate in the list that is the one you added to use for Search Grid to encrypt messages being sent to the CEP server.
24. RIGHT-click on that certificate and choose [Properties](#).
25. On the [General](#) tab, near the top, you will see a [Friendly Name](#) field. This contains the alias to be used by Search Grid to identify that certificate.
 - If there is already a value in the [Friendly Name](#) field, you may keep that, or you may change it to something else.
 - If there is nothing in the Friendly Name field , enter [CEP-Artemis](#) or any name of your choice.
 - Make note of that Friendly Name. You will need it later.
26. Click [OK](#).
27. In the list of certificates, for that certificate, you should now see the pre-existing value or the value you entered under the Friendly Name column. You may now close MMC. If it asks to save the Console settings, you may choose [No](#).
28. Refer to the [Setting CEP Properties In Search Grid](#) section for instructions on how to set properties, and set the following two properties:

- `cep.keystore.alias=TheFriendlyNameForTheCertificateYouJustAdded`
- `cep.keystore.type=Windows-MY`

Setting CEP Properties In Search Grid

If using the default certificate provided by the CEP Server, installed exactly as directed in the [Using The Default CEP Client Certificate](#) section, configuration only requires configuring the mandatory CEP properties. Likewise, if a single custom certificate can be used to authenticate the CEP server and encrypt the data being sent to it. If the desire is to have one custom certificate that establishes the issuer of the custom certificates as a trusted authority, and another certificate that holds the encryption keys needed to communicate with the CEP server, then you *might* need to also set the `cep.keyStoreType` property as explained below.

Like most other Search Grid properties, the CEP properties can be set in each server's `<install-location>\etc\<node-type>\<service-type>-properties.xml` file for each Search Grid service, or `<install-location>\etc\local-properties.xml` of each server running Search Grid services or via global properties set with the "set-gprop" command in Search Grid's [management-console](#), and if set in multiple places, the above order is the order of precedence, highest first, lowest being the global properties. **The one exception for CEP settings is for the Control service, which can only get its settings from `control-service-properties.xml` or `local-properties.xml`, not the global properties.**



If your search grid has many data and/or compute services spread across many servers, the simplest way to configure CEP is to set the settings in the `control-service-properties.xml` or the `local-properties.xml` file on the server(s) that the Control service runs on, and use the global properties to distribute the settings to all the other servers.

- Any text editor, such as Notepad, can be used to edit the properties files, *but whatever editor is used, it should be launched using "Run as administrator" so that it has permissions to write to the file.*
 - When setting a CEP property in any of the aforementioned files, **For all properties mentioned below, if there is an entry in the file with that configuration Key (property name), modify that line; however, unless CEP had been used previously, the lines for the properties will need to be added to the end of the file.**
 - The format for all properties specified in the xml files is:


```
<entry key="KeyNameWithinQuotes">Value</entry>
```
 - To remove a property, delete that line.
- If using the [management-console](#) to set global properties using set-gprop commands, log into the management console as the admin user.
 - Use `set-gprop <property> <value>` to set or change a property.
 - Use `set-gprop <property>` to remove a property.



All property names and values are case-sensitive

Changes to property values will not be recognized until Search Grid services are restarted.

Mandatory Properties To Enable CEP

cep.broker.hostFQDN=FQDN-of-the-CEP-server

Setting this property enables CEP and tells Search Grid the CEP server's Fully Qualified Domain Name.

XML file example:

```
<entry key="cep.broker.hostFQDN">myCEPserver.somesubdomain.company.com</entry>
```

Global properties example:

```
set-gprop cep.broker.hostFQDN myCEPserver.somesubdomain.company.com
```

Optional CEP Properties

The values shown below are the default values used when the property is *not* specified in the application.properties file. Unless there is a site-specific need to set any of these values to custom settings, it is recommended to not set them and let them default.

cep.broker.port=30101

Only if your CEP server uses a custom port number rather than the default port of 30101, use this property to specify that custom port number.

cep.queue.exceptions=VoiceGrid.Exceptions

This specifies the queue name on which the CEP server expects to receive exceptions from Search Grid. Needing to set this is very uncommon.

cep.queue.heartbeat=Heartbeats

This specifies the queue name on which the CEP server expects to receive heartbeat messages from Search Grid. Needing to set this is very uncommon.

cep.heartbeatSeconds=60

This specifies how often Search Grid sends heartbeat messages to the CEP server. Any change here will likely need to be coordinated with CEP server settings that indicate how often to expect heartbeats. Needing to set this is very uncommon. Setting this to 0 will disable heartbeats.

cep.exceptionBacklogMaximum=200000

This specifies the maximum number of exception messages that Search Grid should queue up to send to the CEP Server if the CEP server is down / unreachable. The messages will be sent in order once a connection is (re-)established. If there are more exceptions than this value during the time that Search Grid cannot communicate with the CEP server, the oldest messages will be discarded as new ones are added.

cep.verifyHost=true

This specifies whether to verify the CEP server's hostname. If custom certificates are used that do not include the ability to verify the CEP server's FQDN (Fully Qualified Domain Name), OR if DNS lookup is disabled on the Search Grid server, preventing such verification, then this property can be set to false (lower-case) to disable verification in order to allow Search Grid to permit communication with the CEP server without verification. Needing to set this is very uncommon and should be avoided unless absolutely necessary.



Warning! Setting cep.verifyHost to false reduces the level of security by permitting some other, unknown entity to pose as the CEP server, allowing potentially-sensitive data to be acquired.

cep.keystore.alias=CEP-Artemis

This specifies the alias or Friendly Name used to find the encryption certificate in the keystore. (See [Using The Default CEP Client Certificate](#) and [Using Custom CEP Certificates](#).) Needing to set this is very uncommon.

cep.keystore.type=Windows-ROOT

This specifies where the certificate used for encrypting messages to the CEP server is located. If custom certificates are being used and there are separate certificates for providing the encryption keys and providing a trusted signing authority used to validate the encryption certificate, AND if you do not wish to add the encryption certificate as a trusted root authority, then the encryption certificate can be installed into the Personal certificates for the service user (see [Using Custom CEP Certificates](#)), in which case this will need to be set to `Windows-MY`.

+ There are other values that are allowed for special-use cases for quickly testing certificates without importing them into windows. (See [Properties For Special-Use Test Cases](#).)

Properties For Special-Use Test Cases

When using custom certificates, it may be desirable to test those certificates (especially when testing new certificates that will replace soon-to-expire certificates) quickly, without having to import them in to Windows. **These properties are not intended to be used other than for short-term testing, as they expose their test keystore (and possibly truststore) passwords.**

To test, these value would be set in the `application.properties` during testing, and then deleted from `application.properties` after testing, or in the case of `cep.keyStoreAlias`, `cep.keyStoreType` and `cep.trustStoreType`, set back to their original values if they had been previously set.

- To test a new encryption certificate, that certificate can be placed in an external keystore file. How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.
- To test a new Trusted Root Certification Authority certificate, such as if the encryption certificate being tested requires authority other than one already imported into Windows, or if you need to test a new Trusted Root Certification Authority certificate before replacing an expiring one, that certificate can be placed into an external keystore (referred to as a truststore). How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.
- If there is only one certificate to be used for both encryption and certification, then that keystore can be specified as both the keystore and truststore, but unless you intend to use a Trusted Root Certification Authority certificate that has already been imported into windows, you must set both the keystore and truststore properties, below.

Specifying An External Keystore File

`cep.keystore.path`

Set this to the full path and file name of your test keystore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

cep.keystore.password

Set this to the password for the test keystore. This would have been set when you created the keystore.

cep.keystore.type

Set this to the type of the test keystore — JKS, JCEKS or PKCS12. If not set, but cep.keystore.path is set, this will default to PKCS12

cep.keystore.alias

Set this to the alias for the encryption certificate contained in the keystore

Specifying An External Truststore File

cep.truststore.path

Set this to the full path and file name of your test truststore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

cep.truststore.password

Set this to the password for the test truststore. This would have been set when you created the truststore.

cep.truststore.type

Set this to the type of the test truststore — JKS, JCEKS or PKCS12. If not set, but cep.truststore.path is set, this will default to PKCS12
